BCSE498J- Project - II

PITCH GYM: AI-POWERED INVESTOR SIMULATION SYSTEM FOR STARTUP PITCH TRAINING

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE3962 DIWAN HENIL BHUSHAN

Under the Supervision of

Prof. Sarwesh P

Associate Professor Grade 1

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled "PITCH GYM: AI-POWERED INVESTOR SIMULATION SYSTEM FOR STARTUP PITCH TRAINING" submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Sarwesh P, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

DIWAN HENIL BHUSHAN (22BCE3962)

Date : 30/04/2026

CERTIFICATE

This is to certify that the project entitled "PITCH GYM: AI-POWERED INVESTOR SIMULATION SYSTEM FOR STARTUP PITCH TRAINING" submitted by DIWAN HENIL BHUSHAN (22BCE3962), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date : 29/04/2026

K.B. Jinesh
30/04/2026
Internal Examiner

External Examiner

Head of the Department

VELLORE INSTITUTE OF TECHNOLOGY (VIT)
School of Computer Science & Engineering (SCOPE)
Vellore-632014

ACKNOWLEDGEMENT

I DIWAN HENIL BHUSHAN (22BCE3962) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Sarwesh P, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

EXECUTIVE SUMMARY

This project represents the development of **Pitch Gym**, an online pitch coaching tool for investors that utilizes Real-Time Voice Artificial Intelligence to train founders in developing high-quality pitches. With Pitch Gym, users will be able to rehearse ten types of pitches, including elevator pitches, Venture Capital (VC)-ready pitches, and technical pitches.

With Pitch Gym, users will get simulated live audio conversations with an artificial intelligence investor named "Alex". This virtual assistant will ask challenging questions and provide constructive feedback in response to each of the proposed pitches.

Pitch Gym is a modern Single Page Application (SPA) based on React 19. The application has Tailwind Cascading Style Sheets (CSS) styling and shadcn/ui User Interface (UI) elements. Supabase is leveraged for user authentication and serverless edge functions. Serverless edge functions represent the back-end Application Programming Interface (API) of this application that configures assistants with the help of the Vapi service, receives webhooks on the completion of call, and controls time credits with the help of time-based credits. It is worth noting that one credit corresponds to one minute of practice time and that call durations are controlled with the help of `maxDurationSeconds` Vapi parameter. As a result, users will not be able to abuse their credit limits.

Among other notable features of the platform there is an Idea Bank for storing multiple startup ideas and session context carry-forward feature that enables an assistant to recall previous conversations. In turn, during sessions, users will have access to real-time transcripts of the interaction. Furthermore, structured JavaScript Object Notation (JSON)-based assessment data and skills ratings represented with skill radar charts and trend lines are available in the analytics section. Finally, users will be provided with a pitch history management module that will help users sort past sessions according to ideas.

Large Language Model (LLM) output repair utility helps manage malformed JSON responses generated by voice Artificial Intelligence (AI) model.

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BaaS	Backend as a Service
CRUD	Create Read Update Delete
CSS	Cascading Style Sheets
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NLP	Natural Language Processing
OAuth	Open Authorization
REST	Representational State Transfer
RLS	Row Level Security
RPC	Remote Procedure Call
SDK	Software Development Kit
SLA	Service Level Agreement
SPA	Single Page Application
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
URL	Uniform Resource Locator
VC	Venture Capital
WebRTC	Web Real-Time Communication

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The number of startups globally has been rising over the last decade to 305 million startups per year. The effectiveness of communication plays a crucial role in startup success since, according to Brooks et al. (2014), the quality of the founder's pitch often becomes a determining factor in making a VC decision to invest and has more value than such variables as market size or product development stage. In contrast, most founders, especially the newcomers, do not get access to repeatable pitch training sessions. Conventional ways of developing one's presentation skills such as getting advice from a mentor or participating in a pitch contest are ineffective due to various limitations, including irregularity, difficulty of scheduling, subjectivity of reviewers, and inconsistency of feedback provided Clark (2008).

The recent advances in AI and speech technologies have opened up multiple opportunities for developing efficient voice-based learning systems. LLM conversational agents such as Claude and OpenAI can conduct sophisticated dialogues, ask follow-up questions, and give consistent analytical feedback on user's performance Brown et al. (2020); Ouyang et al. (2022). Moreover, significant progress in Speech-to-Text (STT) and Text-to-Speech (TTS) technologies, represented by the latest developments such as OpenAI Whisper model, enables implementing commercial-scale interactions using one's voice Radford et al. (2023). There are also platforms that facilitate creating fully functional voice agents using existing technologies such as Vapi AI. Thus, developers can create voice-first applications without setting up any infrastructure required for that process Vapi, Inc. (2024).

Web technologies are also advancing in the contemporary world, and it allows creating high-quality websites and web applications. Currently, there are powerful frontend frameworks capable of creating highly interactive SPAs such as React 19, which is used in such famous sites as Instagram and Tesla Meta Platforms, Inc. (2024). Additionally, Backend as a Service (BaaS) platforms like Supabase allow deploying highly interactive web projects based on PostgreSQL databases with authentication, real-time

subscriptions, and serverless edge functions at once Supabase, Inc. (2024).

1.1.1 Problem Domain

The problem space for the development of this project is at the crossroads of entrepreneurship education, conversational AI, and web application development. Entrepreneurs require a solution that allows them to practice their pitching skills and meet these criteria:

1. **On-demand access** — entrepreneurs have the ability to practice whenever they want, rather than only when scheduled.
2. **Realism** — the practice partner is supposed to act as a real investor, putting pressure on the entrepreneur by asking difficult questions.
3. **Structured feedback** — it should be quantified and specific feedback instead of subjective one.
4. **Progressive** — it should measure improvement and adjust to the entrepreneur's skillset.
5. **Diversity** — the entrepreneur should have the chance to practice multiple scenarios, such as elevator pitch, VC pitch, technical pitch, and demo day.

1.1.2 Current Systems and Limitations

There are various options that already exist for providing pitch training to entrepreneurs; however, each of them has its own limitations:

- **Startup accelerator programs** (Y Combinator, Techstars, 500 Global): These offer excellent mentorship opportunities but are extremely selective (less than 3% acceptance rate), geographically confined, and bound to certain schedules, as per Ries (2011).
- **Human pitch coaches**: These are costly ($200–500 per session), have limited availability, and bring subjectivity to the evaluation process.

- **Chatbots** (ChatGPT, Claude): These can mimic the conversations that take place between an investor and a founder; however, they lack the vocal aspect of it. Pitches require more than just speaking — they require the delivery of words, and the ability to be quick on the feet. These elements cannot be tested through text conversation alone.

- **Pitch presentation software** (Pitch.com, Beautiful.ai): These cater to presentations, i.e., slide creation, rather than verbal pitch delivery and interaction with investors.

- **Video recording software**: These allow founders to record themselves while delivering pitches but do not facilitate any kind of feedback mechanism or investor questioning.

1.1.3 Relevant Technologies

Several key technologies used in this project include:

- **Large Language Models (LLMs)**: Foundation models trained using massive text data that simulate human-like interactions. The model relies on LLMs using the Vapi platform to enable investor persona behavior, which involves posing pertinent questions and conducting assessments Brown et al. (2020).

- **Speech-to-Text (STT)**: Automated speech recognition systems that transform oral language into written text. This technology makes it possible for the AI to comprehend what the entrepreneur is saying during the voice pitch presentation Nassif et al. (2019).

- **Text-to-Speech (TTS)**: Neural voice generators that transform AI responses from text format to speech output. This creates an interactive conversation between the startup founder and AI Borsos et al. (2023).

- **Web Real-Time Communication (WebRTC)**: Browser-native technologies for transmitting audio and video information in real-time without plug-ins or extra software Sredojev et al. (2015).

- **React 19**: A declarative framework for JavaScript coding that allows the development of user interfaces concurrently and server-side components Meta Platforms, Inc. (2024).

- **Supabase**: An open-source BaaS provider with PostgreSQL database capabilities, JSON Web Token (JWT)-based authentication, Row Level Security (RLS) policies, and Deno-based edge functions Supabase, Inc. (2024).

1.1.4 Need for the Proposed System

The development of startup ecosystem has led to the genuine need for scalable, high-quality pitch training services. Statistics indicate that over 90 percent of startups fail, and poor communication of the value proposition is consistently cited as a leading cause of funding failure Chen et al. (2009). Conventional approaches to pitch training cannot be scaled to cope with this problem. The problem can be solved through pitch training via Pitch Gym as an ever-ready AI-driven practice partner.

1.2 MOTIVATION

1.2.1 Problem Significance

The capacity to make an impactful investor pitch is among the critical skills that an entrepreneur must master. In just one pitch session, a startup might secure funding to help it sustain and scale its operations. However, the process of preparing an impactful investor pitch still ranks among the least systemized stages in entrepreneurship. Entrepreneurs have unlimited information regarding business models, products, and technology. But pitching training is mostly done through informal channels and is not easily accessible.

1.2.2 Practical Motivation

From a practical standpoint, the motivation for this project is due to the reality that founders, especially those who have a technical background, belong to underserved geographical areas, or come from their first generation of entrepreneurship within their family, often do not have access to mentors and investors who can offer them training

sessions and feedback. Through a voice-based AI interface, one can have that experience anytime and anywhere.

1.2.3 Academic Motivation

From an academic perspective, this project investigates the implementation of recent advances in artificial intelligence technology, namely voice agents powered by large language models, within an educational and vocational training setting. This plays a part in the burgeoning niche of AI-supported education research Zawacki-Richter et al. (2019); Chen et al. (2020) by demonstrating how conversational AI can facilitate structured and domain-relevant assessment of a verbally and interactively oriented skill. Additionally, this project considers practical software engineering problems such as audio streaming, natural language processing, partial grading, and webhook architectures.

1.2.4 Technical Interest

The project involves the integration of many components that pose various technical challenges, such as real-time voice-based AI interaction using WebRTC, JSON data extraction from the output of the LLMs even if the output is corrupted or incorrectly formatted, resource management on the server side using edge functions, and a single-page app.

1.2.5 Innovation Aspect

Pitch Gym introduces several novel aspects compared to existing solutions:

- **Ten specialized pitch types**: Rather than a generic conversation, the system offers ten distinct pitch scenarios, each with a custom AI investor persona and tailored assessment criteria.
- **Session context carry-forward**: Users can optionally carry context from a previous session into a new one, allowing the AI investor to reference past performance and track improvement.
- **Time-based credit system**: Credits are consumed based on actual session duration (1 credit = 1 minute), with server-side enforcement to prevent abuse.

- **Structured multi-metric scoring**: Each session produces a structured assessment with per-category scores (1–5 scale), qualitative analysis, strengths, weaknesses, and actionable improvement suggestions.

1.3 SCOPE OF THE PROJECT

1.3.1 Functional Scope

Functional scope of Pitch Gym covers the following capabilities:

- **User authentication**: Registration and sign in using email/password, Google Open Authorization (OAuth) integration, resetting password using email, and Supabase Auth for session management.
- **Session Management of Pitching**: Choosing a pitch type among ten available choices; choosing a level of difficulty among three options, along with an optional choice of idea; voice conversation with the AI investor; display of live transcript; session timekeeping including countdown and warnings for low time.
- **Assessment and Scoring**: Automatic structured assessment creation by AI after each session; per-metric scoring on a 1–5 scale; an overall rating; analysis of strengths and weaknesses, improvement areas.
- **Idea Bank**: Create Read Update Delete (CRUD) capabilities of multiple startup ideas; linking of session pitches to corresponding ideas.
- **Analytics Dashboard**: Tracking aggregate score trends overtime through line chart; skill radar charts based on assessment categories; distribution stats of pitch types; ability to filter analytics by individual idea.
- **Pitching History**: Listing of past pitching sessions; details view of results per session; moving around the history to select past session.
- **Credit System**: Credits based on time (1 credit = 1 minute); fractional credits (e.g., 4.5 minutes left); ability to buy credits through Razorpay payment gateway; session management on server side.

- **Carry-over of Context**: Optional ability to choose from a list of previous sessions for providing context to the AI investor; preview of context summary before starting a new session.

1.3.2 Non-Functional Scope

- **Performance**: This project aims for sub-second loading times using the optimized bundling and code splitting offered by Vite. Latency is kept low by Vapi's hosting capabilities.
- **Scalability**: The serverless design (Supabase Edge Functions and Vapi cloud) scales automatically depending on demand, requiring no additional infrastructure setup.
- **Security**: Access to all data is regulated by Supabase RLS, allowing only access to users' own data. Tokens used are JWTs that automatically refresh. Deduction of credits is done server-side to avoid any kind of tampering from clients.
- **Usability**: The UI adheres to a modern UI design philosophy featuring a dark theme with good layouts and readable contrasts between colors.
- **Reliability**: LLM parsing supports automatic correction of common JSON problems such as smart quotes and unescaped characters.

1.3.3 Technical Scope

- **Frontend**: React 19 with Vite 7, Tailwind CSS version 4, shadcn/ui libraries, and Recharts for data visualization.
- **Backend**: Supabase (PostgreSQL, Auth, Edge Functions, and RLS).
- **Voice AI**: Vapi AI Web Software Development Kit (SDK) with WebRTC voice communication.
- **Payments**: Razorpay for payments using credit cards.

1.3.4 Project Boundaries

- **Assumptions**:
  - Users are using an updated web browser with access to the microphone.
  - Users have an adequate internet connection for live voice streaming.
  - The Vapi AI platform and Supabase services are operating within the constraints of their Service Level Agreement (SLA)s.
- **Constraints**:
  - The software will not provide video-based training for pitches (only audio).
  - The quality of assessment is dependent on the capabilities of the LLMs used and the prompt engineering for each individual pitch type.
  - The software will not make connections between entrepreneurs and investors (just a practice tool).
  - Session length is limited to credit availability per pitch type (maximum of 3 min for Elevator Pitch, 10 min for Demo Day).

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

This section examines previous works related to the Pitch Gym application by dividing them into four broad categories: artificial intelligence-driven conversational agents, artificial intelligence in education and training, studies on startup pitches, and recent web applications.

2.1.1 AI-Powered Conversational Agents

The transformer model introduced by Vaswani et al. (2017) laid a foundation for the current AI revolution in conversational technologies. By substituting recurrent neural networks with self-attention mechanisms, transformers made parallel processing of sequence data possible, improving efficiency and the quality of training and output. The next major breakthrough in language modeling is attributed to Brown et al. (2020).

This team showed through their experiments with GPT-3 that scaling language models to 175 billion parameters enables few and zero-shot performance in variousNatural Language Processing (NLP) tasks without fine-tuning. It was proven that sufficiently large language models could function as conversational AI systems in different roles (investor persona for Pitch Gym). Thus, with sufficient prompt guidance, they could perform the tasks they were not explicitly trained to do.

Another critical breakthrough comes in the form of aligning language models with human intent in the work of Ouyang et al. (2022). Their experiment with InstructGPT demonstrated that human feedback-based fine-tuning leads to significantly more helpful, harmless, and honest outputs in comparison with base models. This technique is crucial for making sure that Pitch Gym behaves like a professional investor rather than an auxiliary assistant by default.

Concerning speech recognition technology, Radford et al. (2023) introduced Whisper — an pre-trained speech recognition system trained on 680,000 hours of multilingual audio data. Using weak supervision at scale, this system reaches human-level accuracy for diverse accents, environmental noise, and speaking rates. Therefore, it is

feasible to implement real-time voice communication for consumer-grade applications. Similarly, Borsos et al. (2023) proved the possibility of generating high-quality audio outputs using AudioLM, a language modeling-based audio generation system.

Finally, Zhang et al. (2023) introduced SpeechGPT, which represents a new generation of multimodal LLMs with built-in conversational cross-modal capacities. Pitch Gym uses a pipeline approach (STT $\rightarrow$ LLM $\rightarrow$ TTS), but the trends towards developing speech-language models suggest that AI voice capabilities are only going to get better.

2.1.2 AI in Education and Training

A large amount of studies have investigatedapplications of AI in education. Zawacki-Richter et al. (2019) reviewed 146 articles about AI in education and found that there are four main categories of applications: profiling and prediction, assessment/evaluation, adaptive learning systems, and intelligent tutoring systems. Although they found that there was great potential in AI-based assessment tools, the majority of these systems involved text-based interactions. Furthermore, AI-based training systems with voice interfaces were rare.

Chen et al. (2020) offered a broader examination of the uses of AI in education, which were divided into three types: administrative uses, instructional applications, and learning support tools. They pointed out that there is currently a lack of domain-specific and skills-focused training systems, which exactly describes the use case of pitch practice. In other words, the current trend is that AI-based training systems will make expert training available at a much lower cost than human tutors.

Hwang et al. (2020) discussed an ideal scenario for using AI in education, which includes personalization, adaptability, and continuous assessment. In the proposed AI system, personalized practice sessions with structured assessment can be scheduled according to learner needs and preferences. It is worth noting that the pitch practice system developed by the founder is consistent with the proposed framework.

Intelligent tutoring systems that used natural language dialogue to assist users were first developed by Graesser et al. (2004). The authors demonstrated that intelligent conversational tutors could produce learning outcomes as effective as human tutors with appropriate interaction techniques. Pitch Gym takes advantage of such intelligent tutor-

ing technology in the professional setting of public speaking.

In their article, Luckin et al. (2016) made the point that AI is a useful tool for assessment and feedback purposes, not a replacement for humans. In addition, the authors suggested that the best intelligent tutoring systems were designed with a strong emphasis on immediate assessment and feedback. Each session in the Pitch Gym software involves a structured assessment based on multiple metrics.

A review of tutoring systems was conducted by Johnson and Lester (2016), who emphasized the significance of incorporating domain knowledge, pedagogical principles, and adaptability in such systems. Moreover, they noted that voice interaction is another important modality in intelligent tutoring systems, especially in professional settings that involve verbal skills.

2.1.3 Startup Pitch Evaluation and Investor Decision-Making

The literature on how the audience evaluates presentations in pitches helps to lay down the theoretical foundation of the Pitch Gym assessment framework. Brooks et al. (2014) carried out experiments demonstrating how investors' impressions about founder passion and preparation mostly conveyed via verbal delivery play an important part in funding decisions. They matter even more than the quality of a business idea. As a result, the authors' findings highlight the need for proper pitch preparation.

Clark (2008) studied the function of a business pitch within VC investment decisions and discovered that there were three roles of a pitch in the process: information transfer, relationship establishment, and signals of credibility. It means that pitch training should cover all three dimensions, which was taken into account when developing Pitch Gym assessment criteria.

Chen et al. (2009) investigated VC investment criteria at various stages, and revealed that early-stage investors paid more attention to the communication skills of founders and their understanding of the market than to financial aspects. Such conclusions helped develop the types of pitches offered by Pitch Gym and their criteria based on the combination of content-related (such as business model clarity and market research) and delivery-related (such as confidence and communication skills, stress resistance).

Huang and Pearce (2017) researched the concept of entrepreneur-investor relationships from the exchange theory perspective. In particular, the authors emphasized the

importance of building rapport with investors during pitch interactions. The need to create an engaging investor persona ("Alex") who behaves consistently throughout all pitch practices stems from this study.

Ries (2011) introduced the Lean Startup methodology emphasizing the role of validated learning achieved through rapid prototyping and experiments. Pitch Gym offers specific pitch types (Idea Validation and Customer Discovery) designed for founders to prove their ability to validate their initial assumptions regarding the problem, potential customers, and the market.

Blank (2020) formulated the principles of Customer Development framework requiring founders to develop both a product and knowledge about its intended customers simultaneously. One of the Pitch Gym's pitches types reflects those requirements.

2.1.4 Modern Web Application Architecture

The technical structure of the application was based on well-known practices applied in contemporary web application development. In particular, the Representational State Transfer (REST) architectural style was proposed by Fielding (2000) as a conceptual framework, serving as the basis for the API-driven approach applied in Pitch Gym.

Jonas et al. (2019) have described a serverless computing model implemented via Supabase Edge Functions. They outlined the key benefits of the technology: scalability, pay-per-use pricing, and lower complexity compared to traditional systems. Such attributes are useful at the startup stage of development, like Pitch Gym is at.

Sredojev et al. (2015) have reviewed the WebRTC protocol stack enabling audio communication within a browser. The implementation of browser-native audio streaming through Vapi Web SDK is used in Pitch Gym to create low-latency connections between the browser and the voice AI server without additional installations needed.

The approaches suggested by Sommerville (2016) and Pressman and Maxim (2014) are generally applicable to software engineering projects and can be considered as a source of guidance for software design.

2.2 RESEARCH GAP

Several gaps are observed based on the literature review, which are being addressed by Pitch Gym:

- **Lack of voice-based pitch training platforms**: Although there are AI assistants in text (ChatGPT, Claude), which can practice conversations in the form of text for preparing pitches, there is no platform where one can have a real conversation through their voice and learn from there. As pitch delivery is essentially an oral activity, the existing gap prevents founders from gaining essential skills like confidence and pace, which contribute to successful pitching Brooks et al. (2014).

- **Unstructured assessment framework**: The existing pitch training platforms lack feedback from users' practices, while other platforms provide an unstructured feedback system (AI chatbot conversations). Pitch Gym will offer a unique, structured, multi-parameter feedback assessment mechanism that will be applied across all the sessions.

- **Lack of longitudinal tracking and assessment**: Current pitch training solutions do not enable tracking over time and learning about how one performs in particular sessions. Pitch Gym's analytics will enable founders to trace performance trends, category development, and how particular sessions influenced one's future performances.

- **Monotonous pitch formats**: The available pitch training programs provide only one pitch type, i.e., pitching to investors. Pitch Gym offers founders multiple pitch types tailored to specific purposes with unique criteria (Customer Discovery test knowledge, Technical Pitch tests architecture defense).

- **Geographical and cost-related barriers**: Pitches from successful founders in specific fields require expensive services from pitch trainers that operate only within startup hubs.

2.3 OBJECTIVES

The goals of the Pitch Gym project are as follows:

1. The creation of a online platform that would allow founders to practice their investor pitches using voice communication with an investor avatar powered by artificial intelligence.

2. The creation of ten different types of pitch practice sessions with varying investor AI behavior and assessment criteria based on various entrepreneurial skills.

3. Implementing a method of evaluation with a numerical rating (1–5) in several categories, along with a textual analysis and specific recommendations for improvement after each session.

4. Creating a longitudinal analytics system that will help measure founder performance in practice sessions and show improvement over time and identify weak spots.

5. Developing the Idea Bank system, which will help manage sessions for multiple startups' pitches separately.

6. Implementing context carry-over functionality that would allow investor avatars to keep information from the previous practice session, thus maintaining some context.

7. Implementing a time-based credit system that is regulated by the server-side to provide fair consumption of the resources while also enabling fractional billing.

8. Integrating a payment system (Razorpay) to purchase credits for using the website and implementing a webhook system and idempotent credit incrementation.

9. Proving the concept of voice AI usage for professional skill training.

2.4 PROBLEM STATEMENT

Though critical to raising funding for startups through effective pitch delivery skills, founders do not have easy access to tools for practicing their pitch delivery skills through an affordable, repetitive process based around vocal interaction. The existing methods available either involve costly human mentors providing occasional guidance, purely text-based interactions through AI systems that are unable to grade the vocal element, or passive recording systems with no active interaction.

To overcome this problem, this project focuses on creating Pitch Gym, a web application using the Vapi AI voice agent software to conduct virtual pitch sessions involv-

ing ten different types of pitches with interactive, structured feedback and performance tracking through the use of any modern web browser.

2.5 PROJECT PLAN

The Pitch Gym was developed following an iterative Agile methodology Beck et al. (2001) divided into phases:

Table 2.1: Project development phases and timeline

Phase	**Activities**	**Duration**
Phase 1: Research & Planning	Literature review, technology evaluation, system architecture design	2 weeks
Phase 2: Core Infrastructure	Supabase setup, authentication, database schema, Vapi integration	2 weeks
Phase 3: Pitch Session MVP	Single pitch type, voice call flow, transcript display, basic assessment	2 weeks
Phase 4: Assessment System	Structured output parsing, scoring, result display, JSON repair	2 weeks
Phase 5: Multi-Pitch Types	All 10 pitch types with distinct prompts and assessment schemas	2 weeks
Phase 6: Idea Bank & Analytics	Idea CRUD, idea-pitch association, analytics dashboard, charts	2 weeks
Phase 7: Credit System	Time-based credits, Razorpay integration, server-side enforcement	2 weeks
Phase 8: Polish & Context	Context carry-forward, UI refinements, spinner components, testing	1 week
Phase 9: Documentation	Report writing, code documentation, deployment guide	1 week

Each phase included requirements analysis, implementation, testing, and review cycles. The project used Git for version control and the Supabase dashboard for database management and edge function deployment.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

Table 3.1 outlines the functional requirements of the Pitch Gym system., categorized by subsystem.

Table 3.1: Functional requirements specification

ID	Category	Requirement
FR-01	Authentication	The system shall allow users to sign-up with email and password
FR-02	Authentication	The system shall support Google OAuth login
FR-03	Authentication	The system shall provide password reset via email
FR-04	Authentication	The system shall maintain authenticated sessions using JWT tokens
FR-05	Pitch Session	The system shall offer 10 distinct pitch types for selection
FR-06	Pitch Session	Each pitch type shall display a description and tested skill tags
FR-07	Pitch Session	The system shall support 3 difficulty levels (Conservative, Balanced, Aggressive)
FR-08	Pitch Session	The system shall establish a real-time voice connection with the AI investor
FR-09	Pitch Session	The system shall display a live transcript during the session
FR-10	Pitch Session	The system shall provide microphone mute/unmute controls

Continued on next page

Table 3.1 – continued from previous page

ID	Category	Requirement
FR-11	Pitch Session	The system shall display a countdown timer during active sessions
FR-12	Pitch Session	The system shall show warnings at 2 minutes and 1 minute remaining
FR-13	Pitch Session	The system shall automatically end the session when time expires
FR-14	Assessment	The system shall generate a structured assessment after each session
FR-15	Assessment	The assessment shall include per-metric scores on a 1–5 scale
FR-16	Assessment	The assessment shall include qualitative analysis for each metric
FR-17	Assessment	The assessment shall identify strengths and weaknesses
FR-18	Assessment	The assessment shall provide actionable improvement suggestions
FR-19	Assessment	The system shall handle malformed LLM JSON output gracefully
FR-20	Idea Bank	The system shall allow users to create, read, update, and delete startup ideas
FR-21	Idea Bank	The system shall allow associating pitch sessions with specific ideas
FR-22	Idea Bank	The system shall filter pitch history and analytics by idea
FR-23	Context	The system shall display previous sessions for a selected idea
FR-24	Context	The system shall allow selecting one previous session for context carry-forward

Continued on next page

Table 3.1 – continued from previous page

ID	Category	Requirement
FR-25	Context	The system shall preview the context summary before session start
FR-26	Context	The AI investor shall reference carried-forward context during the session
FR-27	Credits	The system shall track user credits in minutes with fractional precision
FR-28	Credits	The system shall deduct credits based on actual session duration
FR-29	Credits	The system shall enforce pitch-type duration caps (3 min Elevator, 10 min Demo Day)
FR-30	Credits	The system shall enforce server-side maximum call duration
FR-31	Credits	The system shall block session start when credits are zero
FR-32	Payments	The system shall create Razorpay payment orders for credit purchases
FR-33	Payments	The system shall verify payment signatures cryptographically
FR-34	Payments	The system shall add credits idempotently after successful payment
FR-35	Analytics	The system shall display score trends over time as line charts
FR-36	Analytics	The system shall display skill proficiency as radar charts
FR-37	Analytics	The system shall display pitch type distribution statistics
FR-38	Analytics	The system shall support filtering analytics by idea

Continued on next page

Table 3.1 – continued from previous page

ID	Category	Requirement
FR-39	History	The system shall list all past sessions with scores and ratings
FR-40	History	The system shall show the associated idea for each session
FR-41	History	The system shall provide detailed result view for each session

3.1.2 Non-Functional Requirements

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Pages shall load within 2 seconds on a basic broadband connection
NFR-02	Performance	Voice latency shall remain below 500ms for conversational flow
NFR-03	Scalability	The system shall support multiple concurrent users without manual scaling
NFR-04	Security	All user data shall be protected by RLS policies
NFR-05	Security	Payment verification shall use HMAC-SHA256 signature validation
NFR-06	Security	Credit operations shall be enforced server-side, not client-side
NFR-07	Usability	The UI shall be responsive in desktop and mobile viewports
NFR-08	Reliability	LLM output parsing shall recover from common JSON errors
NFR-09	Availability	The system shall target 99.9% uptime via managed cloud services
NFR-10	Maintainability	Code shall follow component-based architecture with separation

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

Feasibility (technical) was assessed from three different angles:

1. **Voice AI Integration**: Vapi AI is a production-grade voice agent framework that provides an integrated STT, LLM processing, and TTS pipeline through a unified API interface. This includes customizable system prompts, structured response format, and WebRTC media streaming capabilities. All these fulfill the necessary criteria for generating a realistic simulation of investors.

2. **Real-time Communication**: WebRTC is a native technology built into modern web browsers that allows peer-to-peer audio communication with latency less than 300 milliseconds. WebRTC session management is handled efficiently using the Vapi Web SDK.

3. **Serverless Back-end Infrastructure**: Supabase Edge Functions built on top of Deno allow serverless computing for implementing authentication proxying, webhook handling, and managing credits. The managed PostgreSQL database meets the data model requirements and is secured using RLS.

3.2.2 Economic Feasibility

The project relies on free-tier and pay-per-use services to keep fixed costs low:

Table 3.3: Cost analysis of platform services

Service	**Pricing Model**	**Estimated Monthly Cost**
Supabase (Free tier)	500MB database, 50K auth users	$0
Vapi AI	Per-minute voice call pricing	$0.10–0.15 per minute
Vercel/Netlify hosting	Free tier for static sites	$0
Razorpay	2% per transaction	Variable

Because credits are time-based, Vapi's per-minute charges are passed directly to users. This makes the unit economics self-sustaining.

3.2.3 Operational Feasibility

There is no need to manage the servers within the serverless approach. Supabase will be responsible for managing databases, backup services, authentication processes, and edge functions. Meanwhile, Vapi will handle voice AI infrastructure, such as model hosting, audio processing, and scalability. Hence, even a minimal team or an individual can run the application.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Requirements

Table 3.4: Client-side hardware requirements

Component	**Minimum Specification**
Processor	Dual-core 1.6 GHz or equivalent
RAM	4 GB
Network	5 Mbps broadband connection
Audio	Built-in or external microphone
Browser	Firefox 90+, Chrome 90+, Edge 90+, Safari 15+

3.3.2 Software Stack

Table 3.5 details the complete software stack used in the Pitch Gym platform.

3.3.3 Database Schema

The application uses the following primary database tables:

3.3.4 API Endpoints

The system exposes functionality through five Supabase Edge Functions:

Table 3.5: Software technology stack

Layer	**Technology**	**Version**	**Purpose**
Frontend Framework	React	19.2.0	Component-based UI library
Build Tool	Vite	7.2.4	Development server and bundler
Styling	Tailwind CSS	4.1.18	CSS styling framework
UI Components	shadcn/ui	Latest	Accessible component primitives
Routing	react-router-dom	7.12.0	Client-side navigation
Charts	Recharts	3.8.0	Data visualization (line, radar, bar)
Icons	lucide-react	0.562.0	Scalable Vector Graphics
Voice AI	Vapi Web SDK	2.5.2	Real-time voice agent integration
Backend	Supabase	Latest	BaaS platform
Database	PostgreSQL	15+	Relational data storage
Authentication	Supabase Auth	Latest	JWT-based auth with OAuth
Edge Functions	Deno Runtime	Latest	Serverless TypeScript functions
Payments	Razorpay	2.9.6	Payment gateway integration

Table 3.6: Database table descriptions

Table	**Description**
`user_credits`	Stores per-user credit balance as `numeric(10,2)` for fractional minute support. Primary key is `user_id`; foreign key to auth.users.
`user_pitches`	Records all pitch sessions including call metadata, transcript, structured output, recording path, duration, credits used, and idea association.
`ideas`	User-created startup ideas with name, description, and timestamps. Used to group pitch sessions by idea.
`payments`	Razorpay payment records with order ID, payment ID, verification status, and credit addition tracking for idempotent processing.

Table 3.7: Supabase Edge Function endpoints

Function	Method	Description
`smooth-task` (Vapi Proxy)	POST	Authenticates user via JWT, checks credit balance, selects assistant by pitch type, configures `maxDurationSeconds` and context overrides, proxies request to Vapi API
`end-of-call-webhook`	POST	Receives Vapi end-of-call report, fetches full call data with retries, stores recording to Supabase Storage, saves structured output to database, deducts time-based credits
`create-order`	POST	Creates Razorpay payment order, maps price tiers to credit amounts, inserts pending payment record
`verify-payment`	POST	Verifies Razorpay payment signature using HMAC-SHA256, marks payment as verified, adds credits idempotently
`razorpay-webhook`	POST	Handles Razorpay `payment.captured` webhook events, verifies webhook signature, calls `process_payment` Remote Procedure Call (RPC)

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Pitch Gym uses a three-level architecture adapted for serverless deployment. Figure 4.1 shows the overview (high-leve)l system architecture.

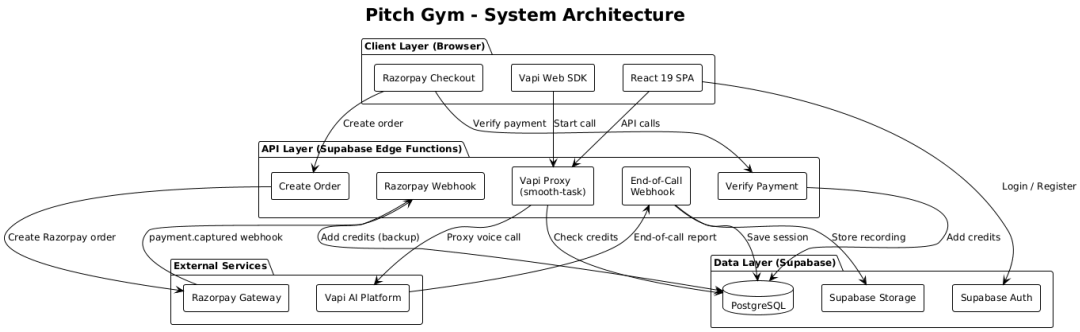

Figure 4.1: System architecture of Pitch Gym (High-Level)

Architecture contains four layers:

1. **Client Layer**: React 19 SPA executing in the user's browser. It manages all UI rendering, state management, voice capture using the Vapi Web SDK, and Razorpay checkout processes.

2. **API Layer**: Five Supabase Edge Functions running in the Deno environment. They include authentication proxy, voice AI configuration, webhooks processing, and payment-related functions. They represent the server-side layer where business logic is applied.

3. **External Services Layer**: Third-party systems such as Vapi AI voice agent platform and Razorpay payments system. They communicate only with the server-side edge functions.

4. **Data Layer**: PostgreSQL database managed by Supabase with RLS policies. Supabase Auth provides identity management and Supabase Storage handles audio recordings.

4.2 USE CASE DIAGRAM

Figure 4.2 presents the use case diagram with all primary interactions between the Founder (user), the Vapi AI platform, and the Razorpay payment gateway.

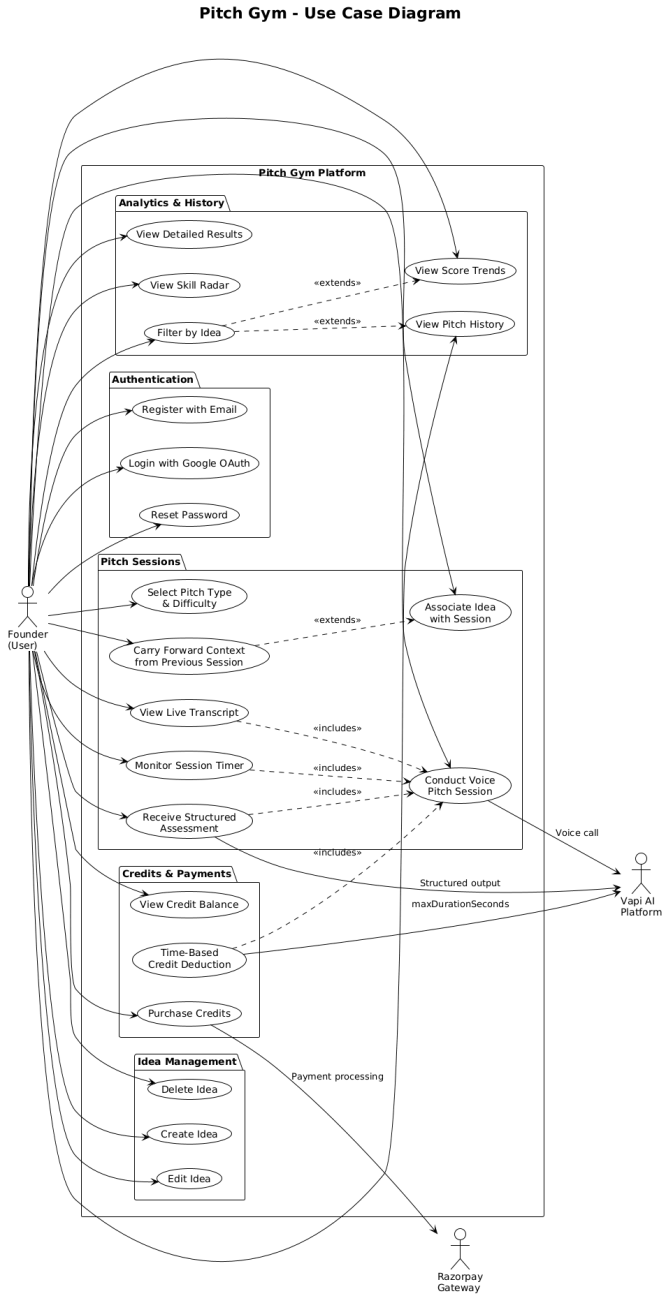

Figure 4.2: Use case diagram of Pitch Gym

21 use cases can be found under the following five categories: Authentication (registration, logging in, resetting password), Pitch Sessions (configuration, running, transcript view, assessment), Idea Management (CRUD functions), Analytic History (trends, radar, filtering), and Credit and Payments (balance, credit purchase, debit).

4.3 ENTITY RELATIONSHIP DIAGRAM

Figure 4.3 illustrates the entity relationship diagram for the database. The database has six primary tables. Foreign key relationships are enforced by PostgreSQL, and access is controlled through RLS policies.

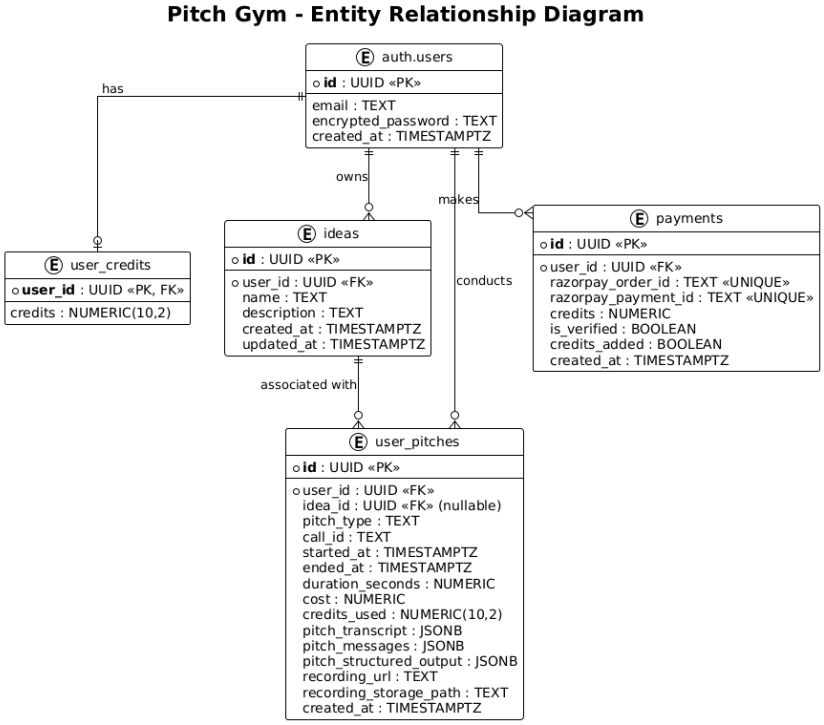

Figure 4.3: Entity relationship diagram

Important relations are as follows:

- A record in `auth.users` has one `user_credits` (1:1).
- A user has many ideas (1:N).
- A user has many `user_pitches` (1:N).
- An idea has many pitches (1:N, Nullable).
- A user has many payments (1:N).
- A pitch has one `pitch_finances` record which has information on Vapi API cost (1:1).

4.4 CLASS DIAGRAM

Figure 4.4 is the component class diagram. It shows the major React page components, utility libraries, hooks, and their dependencies.

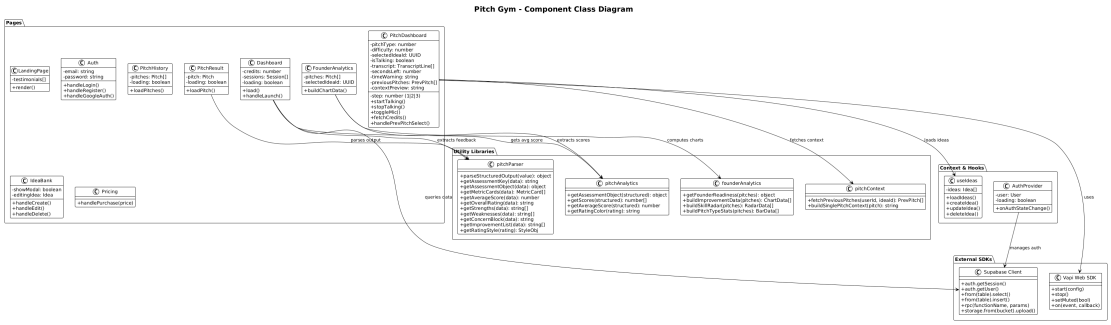

Figure 4.4: Component class diagram

Figure illustrating the dependency of page modules on utility modules. `PitchResult` relies on `pitchParser` for output formatting. `FounderAnalytics` depends on `founderAnalytics` and `pitchAnalytics` for calculation of chart data. `PitchDashboard` incorporates the Vapi Web SDK and `pitchContext` to manage voice sessions and context propagation.

4.5 DATA FLOW DIAGRAMS

4.5.1 Level 0: Context Diagram

Figure 4.5 shows the context diagram, representing Pitch Gym with its external entities: the Founder, Vapi AI Platform, Razorpay Gateway, and Supabase Database.

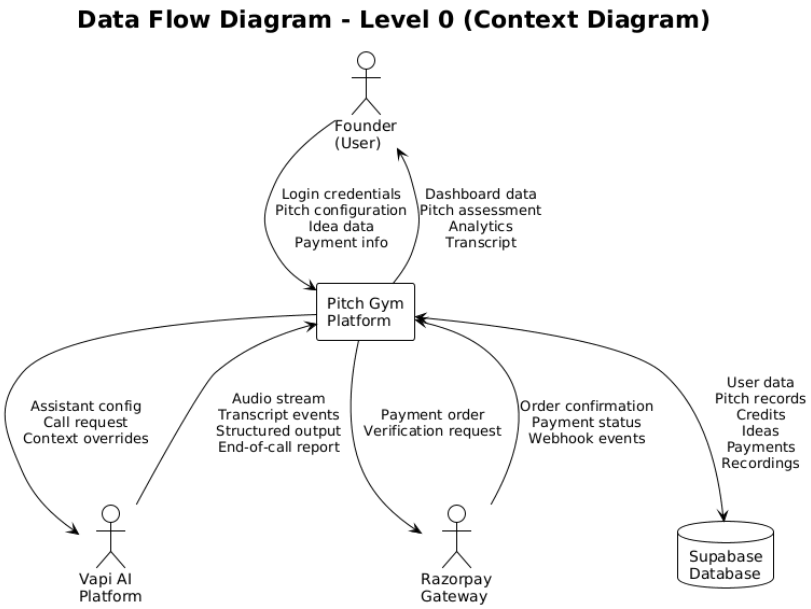

Figure 4.5: Context Diagram

4.5.2 Level 1: Major Processes

Figure 4.6 breaks down the platform into seven different processes: Authentication System (1.0), Pitch Session Manager (2.0), Assessment Processor (3.0), Idea Manager (4.0), Credits Manager (5.0), Payment Processor (6.0), and Analytics Engine (7.0). The data stores include: users, user_pitches, ideas, user_credits, payments, and recordings.

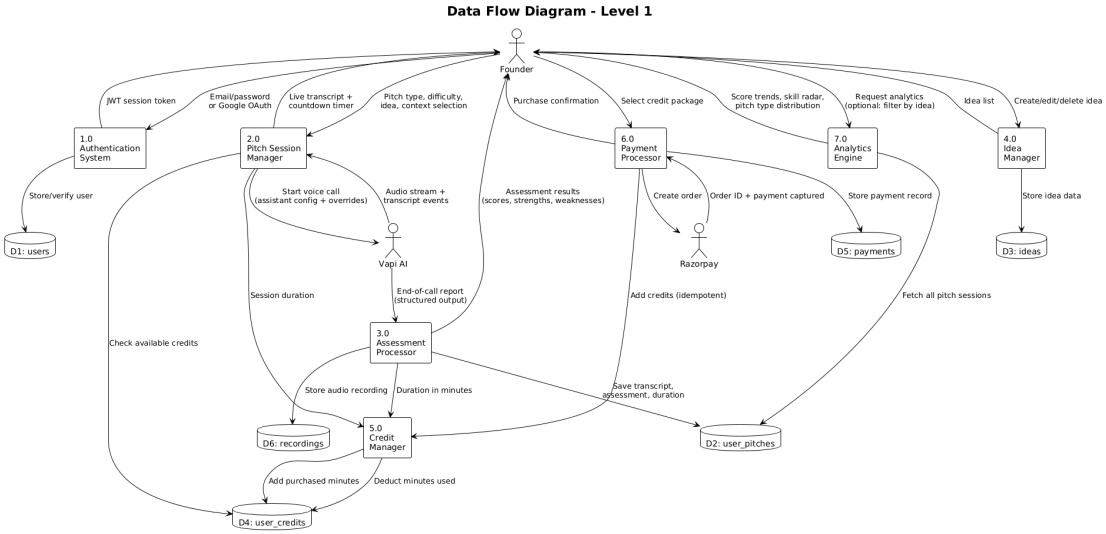

Figure 4.6: Data flow diagram – Level 1

4.5.3 Level 2: Pitch Session Manager

Figure 4.7 presents the breakdown of Process 2.0 (Pitch Session Manager) into seven sub-processes: Session Configurator (2.1), Credit Validator (2.2), Context Builder (2.3), Vapi Proxy (2.4), Call Manager (2.5), Transcript Renderer (2.6), and Countdown Timer (2.7).

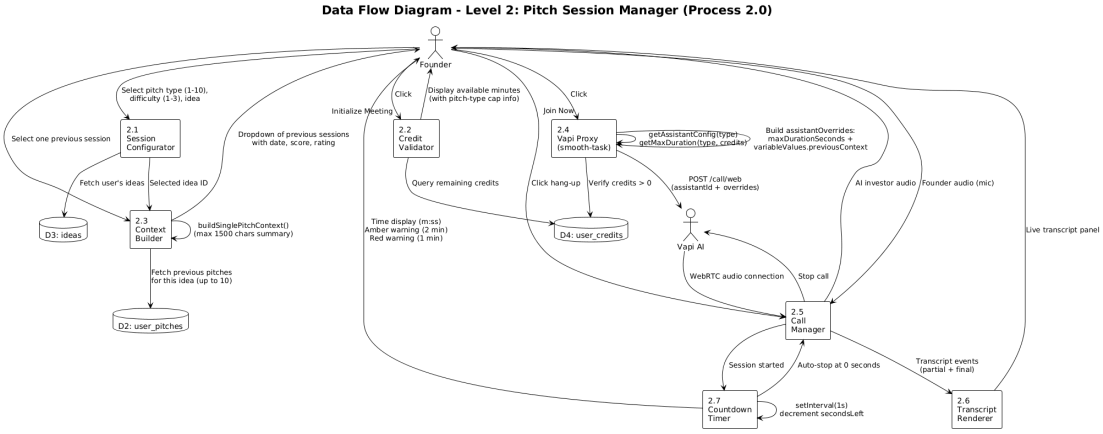

Figure 4.7: Data flow diagram – Level 2: Pitch Session Manager (Process 2.0)

4.6 SEQUENCE DIAGRAMS

4.6.1 Pitch Session Sequence

Figure 4.8 is the sequence diagram for one full pitch session, covering interactions across all system components from configuration through post-call assessment.

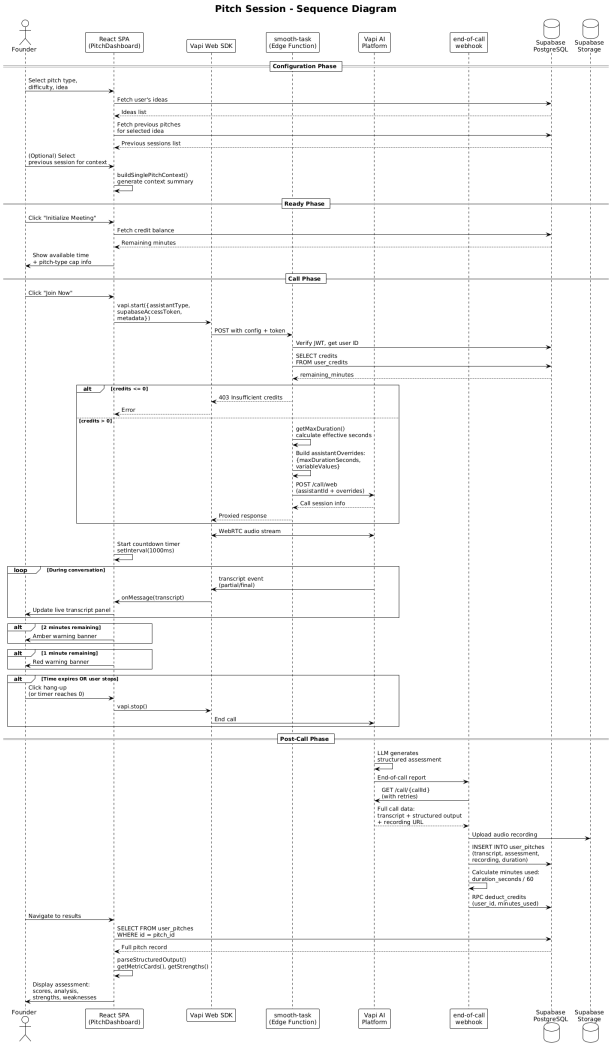

Figure 4.8: Sequence diagram – pitch session lifecycle

There are three clear stages within the process. The Configuration stage involves choosing the pitch type and establishing the context. The Call stage consists of Vapi Proxy authentication, WebRTC Audio, live transcription, and timer alerts. The Post-Call stage manages structured output parsing, saving recordings, deducting credits, and displaying results.

4.6.2 Authentication Sequence

Figure 4.9 shows the authentication sequence for both email/password and Google OAuth flows.

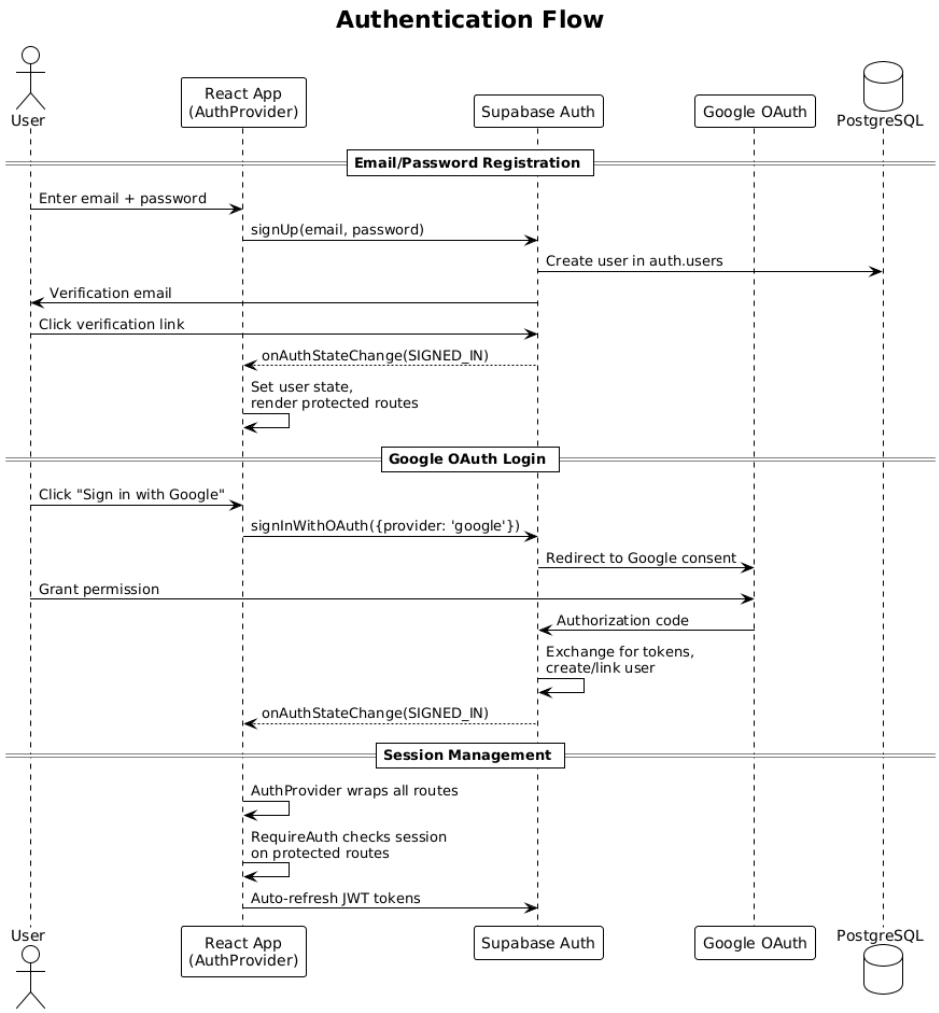

Figure 4.9: Sequence diagram – authentication flow

Authentication system uses Supabase Auth with two sign-in methods:

1. **Email/Password**: Standard registration and login with email verification. Password reset is handled via a magic link sent to the user email that redirects to the `/reset-password` route.

2. **Google OAuth**: One-click sign-in using Google accounts. Supabase handles the OAuth 2.0 flow, token exchange, and profile creation.

The React application wraps all routes in an `AuthProvider` context that listens to Supabase's `onAuthStateChange` event. Protected routes are guarded by a `RequireAuth` component that redirects unauthenticated users to the login page.

4.6.3 Payment Processing Sequence

Figure 4.10 shows the dual-verification payment processing architecture.

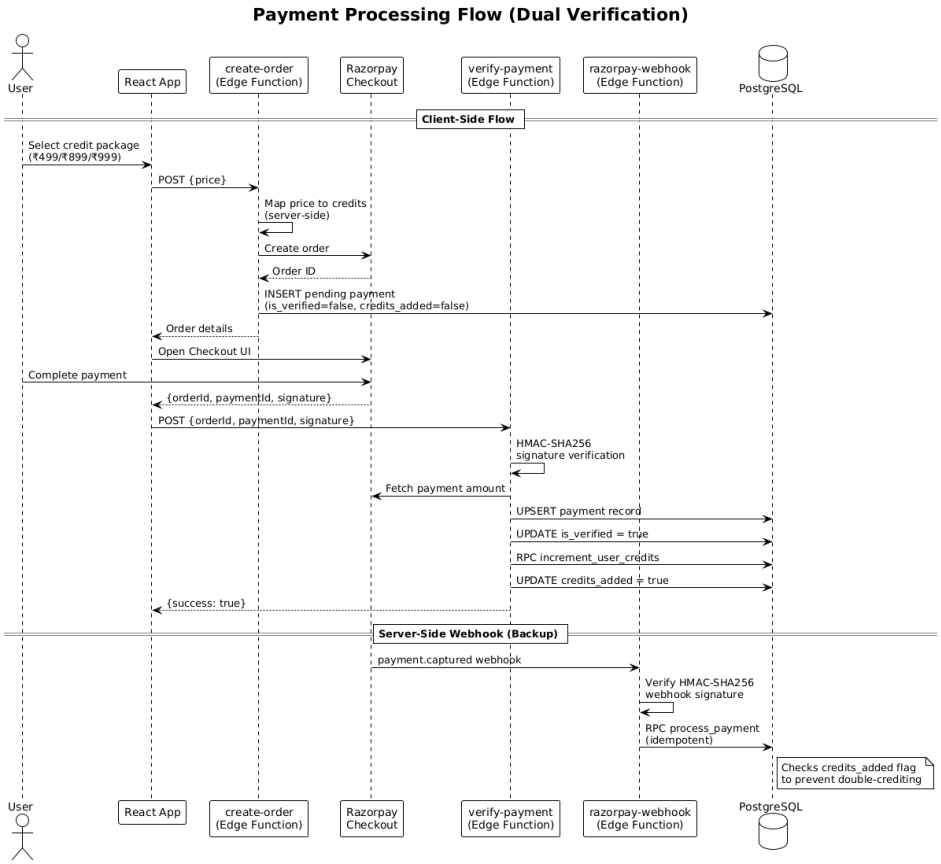

Figure 4.10: Sequence diagram – payment processing (dual verification)

The payment system employs two different verification paths to guarantee reliability:

1. **Client-side verification**: User selects credit package → `create-order` edge function generates Razorpay order → Razorpay checkout UI accepts payment → `verify-payment` edge function validates the signature and applies the credits.

2. **Server-side webhook verification**: Razorpay triggers a `payment.captured` webhook → `razorpay-webhook` edge function verifies the HMAC-SHA256 signature and calls the `process_payment` Remote Procedure Call (RPC) to add the credits as an alternative verification path.

Both the above paths are idempotent in nature. Since the payments table monitors if credits have been added to a particular payment, there is no chance of duplicate credits being applied in either case.

4.7 STATE DIAGRAM

Figure 4.11 displays the state machine for the `PitchDashboard` component, which manages the three-step pitch session interface.

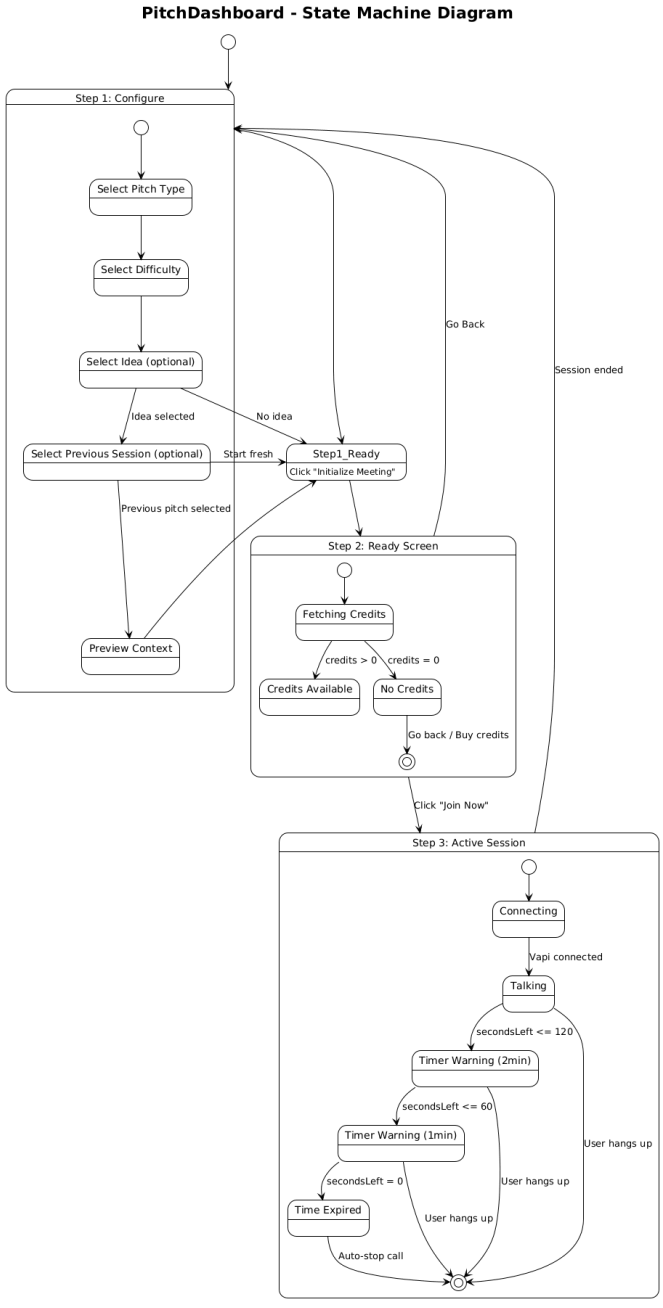

Figure 4.11: State diagram – PitchDashboard component

The component has three main states:

- **State 1 (Configure)**: The user picks the pitch type, difficulty level, the idea behind it, and optional context. This state advances to State 2 by pressing “Initialize

Meeting."

- **State 2 (Ready)**: It shows credits available and the maximum number of pitches allowed. It blocks if the credits are equal to zero. It advances to State 3 by pressing "Join Now."
- **State 3 (Session Active)**: This state controls the entire voice calling process with different states including Connecting, Talking, Time Warning (Amber: 2 minutes, Red: 1 minute), and Time Expired.

4.8 USER FLOW DIAGRAM

Figure 4.12 shows an example of the complete user flow in the Pitch Gym system, from initial landing page visit across all major features.

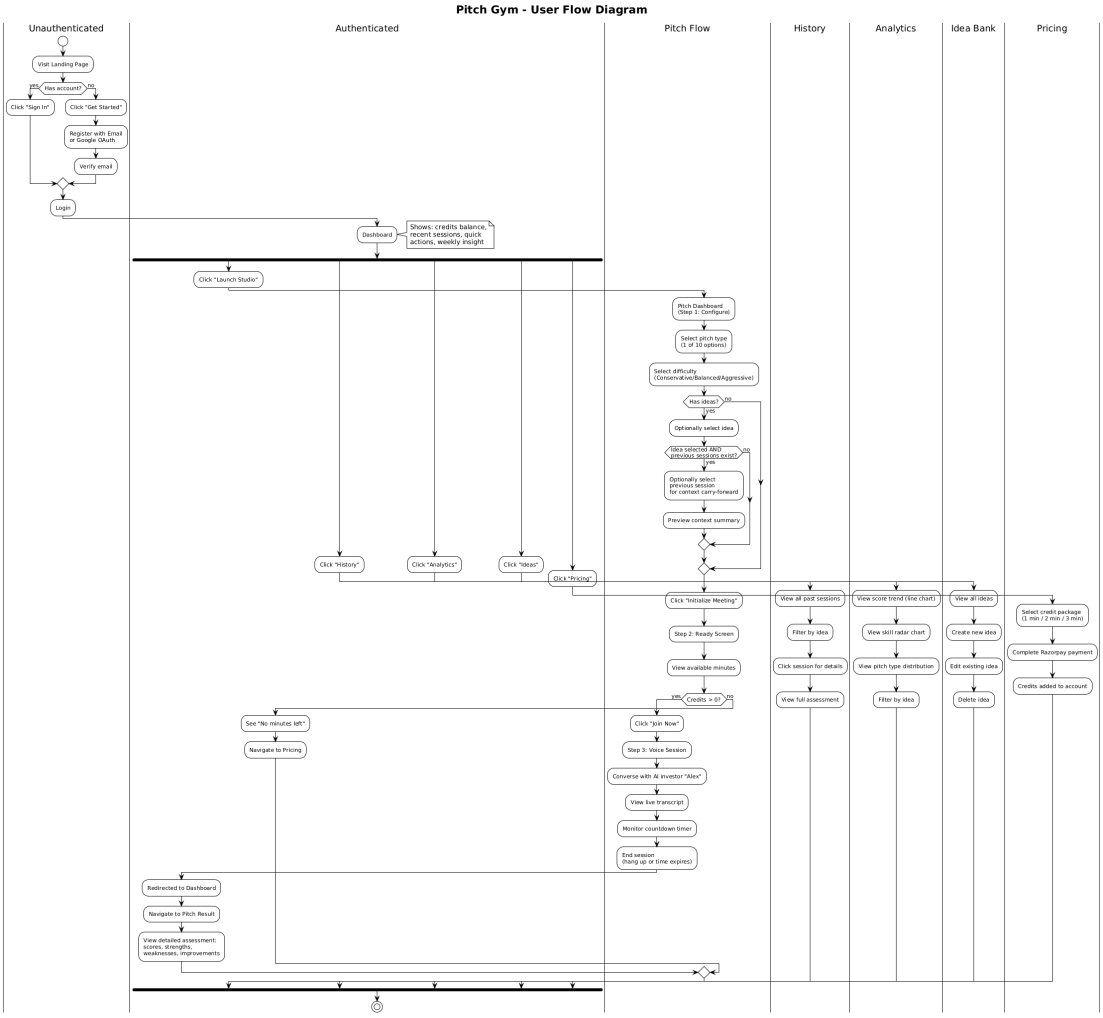

Figure 4.12: User flow diagram

Following authentication, the user proceeds to the Dashboard screen. From here, they can go through five main flows: Pitch Studio (configuring and conducting voice calls), Pitch History (reviewing previous calls), Analytics (analyzing performance), Idea Bank (managing startup ideas), and Pricing (buying credits). The most complicated flow is the pitch flow. This flow uses a three-part wizard process (configure $\rightarrow$ ready $\rightarrow$ active call), which can branch into selecting an idea or carrying forward context.

4.9 DETAILED SUBSYSTEM DESIGN

4.9.1 Vapi Proxy Design

The Vapi Proxy edge function (`smooth-task`) acts as the orchestrator for voice calls:

1. **Authentication**: Retrieves the Supabase access token and validates it with Supabase Auth to get a trusted user ID.

2. **Credit Check**: Checks the `user_credits` record to see how many minutes are left. If the credit amount is equal to or below zero, returns HTTP 403.

3. **Assistant Assignment**: Converts the integer value of `assistantType` (1 to 10) into an equivalent Vapi assistant ID. There are ten preset assistants in Vapi that correspond to each pitch type, including a unique system prompt and output schema.

4. **Duration Calculation**: Determines the max call duration based on the lower value between user credits and the pitch type limit (3 minutes for Elevator Pitch, 10 minutes for Demo Day). Converts to seconds to use as the `maxDurationSeconds` override.

5. **Previous Context**: Uses a `previousContext` string in the metadata to pass it as `variableValues` override. The Vapi assistant will use a Handlebars template {{`#if previousContext`}} in its system prompt to inject the `previousContext` content.

6. **Proxying**: Sends the constructed request to the Vapi API and forwards its response back to the client.

4.9.2 Credit System Design

The credit system utilizes a time-based approach:

- **Storage**: Credits are stored using `numeric(10,2)` data type in PostgreSQL, supporting fractional numbers (e.g., 4.37 minutes).
- **Deduction**: Upon call completion, the end-of-call webhook calculates `durationSeconds/60` and invokes the `deduct_credits` RPC method, where credits are subtracted from the user's balance in one atomic transaction.
- **Time enforcement server-side**: The Vapi proxy specifies `maxDurationSeconds` for each call. The voice platform will hang up the call in case of exceeding time, regardless of whether the client-side timer is circumvented.
- **Timer client-side**: A countdown timer provides feedback to the user by issuing notifications when there are 2 and 1 minutes left before finishing the call.

4.9.3 Structured Assessment Design

Each Vapi assistant has a structured output schema that the LLM fills in at the end of the conversation. The schema follows the same pattern across all ten pitch types:

```
{
  "<pitch_type>_assessment": {
    "metric_1": { "score": 1-5, "analysis": "..." },
    "metric_2": { "score": 1-5, "analysis": "..." },
    ...
  },
  "overall_rating": "Excellent|Good|Fair|Poor",
  "strengths": [{ "point": "..." }, ...],
  "weaknesses": [{ "point": "..." }, ...],
  "biggest_concern": "...",
  "improvement_suggestions": ["...", ...]
}
```

This structure is managed by the client-side parser (`pitchParser.js`), which employs several recovery methods:

- Automatic detection of the test key using suffix pattern (`*_assessment`).
- Fixing JSON structure issues caused by LLM output, including the correction of smart quotes, removal of extraneous commas, and proper escaping of characters.
- Graceful degradation, whereby non-existent data fields produce null values.

4.9.4 Context Carry-Forward Design

The context carry-over functionality provides continuity between pitch sessions:

1. The system searches up to 10 previous pitch sessions related to the selected idea through the `user_pitches` database.
2. The user chooses one previous session based on pitch type, date, time, rating, and average score provided in a drop-down list.
3. The `buildSinglePitchContext` function builds a compact summary (up to 1,500 characters) including the rating, strengths, and weaknesses of the previous pitch session.
4. The summary is used as the `previousContext` attribute in the session data object, which is injected into the system prompt template via the proxy service using template variables.
5. As a result, the AI investor can assess th previous performance of the entrepreneur. This method adds reality to the coaching process.

4.9.5 Component Architecture

The React application is structured using the following component tree:

- `App.jsx`: Root component with route definitions using `createBrowserRouter`.
- `AuthProvider`: Context provider wrapping all routes, managing authentication state.

- `AppLayout`: Shared layout component with navigation.
- `RequireAuth`: Route guard component that redirects unauthenticated users.
- Page components:
  - `LandingPage`: Public marketing page.
  - `Auth`: Login/registration page.
  - `Dashboard`: User home with credits, recent sessions, and quick actions.
  - `PitchDashboard`: Multi-step pitch session interface (configure $\rightarrow$ ready $\rightarrow$ call).
  - `PitchHistory`: Chronological session listing with idea grouping.
  - `PitchResult`: Detailed assessment view for a single session.
  - `FounderAnalytics`: Charts and analytics dashboard.
  - `IdeaBank`: Idea CRUD management interface.
  - `Pricing`: Credit purchase page with Razorpay integration.
- Utility libraries:
  - `pitchParser.js`: LLM output parsing and JSON repair.
  - `pitchAnalytics.js`: Score extraction and aggregation.
  - `pitchContext.js`: Previous session fetching and context building.
  - `founderAnalytics.js`: Advanced analytics computation.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 DEVELOPMENT METHODOLOGY

This project utilized an Agile software development approach, which is characterized by iterations, as described by Beck et al. (2001) and Schwaber and Sutherland (2020). For each feature added, the following process took place:

1. **Requirement Analysis**: Determine the user stories and criteria for acceptance for the feature.
2. **Design**: Define the structure of components, data flow, and API communication.
3. **Implementation**: Develop the React components for the frontend and Edge functions for the backend.
4. **Manual Testing**: Test the feature through browser testing.
5. **Iteration**: Repeat the previous steps until the desired result is achieved.

Git was used for version control; major features were implemented in feature branches, while minor ones were directly committed into the master branch.

5.2 SYSTEM TESTING

5.2.1 Unit-Level Verification

Key utility functions were verified through manual and exploratory testing. The functions tested are listed below:

5.2.2 Integration Tests

Integration tests checked the complete communication between components.

5.2.3 User Acceptance Tests

User acceptance tests were run for confirmation that Pitch Gym meets end-user expectations.

Table 5.1: Unit tests for utility functions

Function	Test Scenario	Expected Result	Status
`parseStructuredOutput`	Valid JSON string input	Returns parsed object	Pass
`parseStructuredOutput`	Already-parsed object input	Returns object as-is	Pass
`parseStructuredOutput`	Malformed JSON with smart quotes	Repairs and parses successfully	Pass
`parseStructuredOutput`	JSON with trailing commas	Repairs and parses successfully	Pass
`parseStructuredOutput`	Null or undefined input	Returns null	Pass
`getAssessmentKey`	Object with `vc_pitch_assessment`	Returns `vc_pitch_assessment`	Pass
`getAssessmentKey`	Object with no assessment key	Returns null	Pass
`getAssessmentKey`	Object with custom `*_assessment`	Returns dynamic key	Pass
`getAverageScore`	Assessment with scores [3, 4, 5, 4]	Returns 4.0	Pass
`getAverageScore`	Assessment with no numeric scores	Returns null	Pass
`getMaxDuration`	Elevator Pitch (type 2), 5 min credits	Returns 180 (3 min cap)	Pass
`getMaxDuration`	Demo Day (type 7), 5 min credits	Returns 300 (5 min, capped by credits)	Pass
`getMaxDuration`	VC Ready (type 3), 5 min credits	Returns 300 (no cap, full credits)	Pass
`getMaxDuration`	Any type, 0 credits	Returns 0	Pass
`formatTime`	125 seconds	Returns “2:05”	Pass
`formatTime`	0 seconds	Returns “0:00”	Pass
`formatTime`	null input	Returns “–:–”	Pass
`buildSinglePitchContext`	Pitch with strengths and weaknesses	Returns formatted summary $\leq$ 1500 chars	Pass
`buildSinglePitchContext`	Pitch with null raw data	Returns null	Pass

5.3 TESTING SUMMARY

All 35 test cases across three categories passed. These results confirm the correctness and usability of the Pitch Gym platform.

Table 5.2: Integration tests

ID	Test Scenario	Expected Behavior	Status
IT-01	User sign-up with email/password	Account saved and created, user redirected to dashboard, credits initialized	Pass
IT-02	Google OAuth login	OAuth flow completes, user session established	Pass
IT-03	Start pitch session with valid credits	Vapi proxy authenticates, selects assistant, call starts	Pass
IT-04	Start pitch session with zero credits	Proxy returns 403, UI shows "no credits" message	Pass
IT-05	Complete pitch session	Webhook saves transcript, assessment, and recording to database	Pass
IT-06	Credit deduction after session	Credits decremented by actual duration (fractional)	Pass
IT-07	Razorpay payment flow	Order created, payment captured, credits added idempotently	Pass
IT-08	Duplicate payment webhook	Credits added only once (idempotency check)	Pass
IT-09	Context carry-forward	Previous session context included in AI investor's awareness	Pass
IT-10	Idea-pitch association	Pitch saved with correct `idea_id`, visible in filtered history	Pass

Table 5.3: User acceptance test cases

ID	Test Scenario	Expected Behavior	Status
UAT-01	New user onboarding	User can register, see dashboard, and start first session within 2 minutes	Pass
UAT-02	Pitch type selection	All 10 pitch types display with descriptions and skill tags	Pass
UAT-03	Voice conversation quality	AI responds naturally, asks relevant follow-up questions	Pass
UAT-04	Assessment comprehensiveness	Results show per-metric scores, analysis, strengths, and improvements	Pass
UAT-05	Timer accuracy	Countdown matches actual session duration within 2 seconds	Pass
UAT-06	Timer warnings	Amber warning at 2 min, red warning at 1 min, auto-stop at 0	Pass
UAT-07	Analytics visualization	Charts render correctly with data from multiple sessions	Pass
UAT-08	Idea management	Create, edit, delete ideas; sessions correctly grouped by idea	Pass
UAT-09	Context carry-forward User Experience	Dropdown shows previous sessions with date/time, preview visible	Pass
UAT-10	Credit display accuracy	Dashboard shows fractional credits matching database value	Pass

Table 5.4: Overall testing

Category	Total	Passed	Pass Rate
Unit-Level Verification	15	15	100%
Integration Testing	10	10	100%
User Acceptance Testing	10	10	100%
Total	**35**	**35**	**100%**

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 FRONTEND IMPLEMENTATION

6.1.1 Application Routing

React-router-dom version 7, along with `createBrowserRouter`, is used for declaring routes. An instance of `RequireAuth` will be placed on all routes requiring authentication to validate that there is an active user session.

```
const router = createBrowserRouter([
  {
    element: <AppLayout />,
    children: [
      { path: "/", element: <LandingPage /> },
      { path: "/auth", element: <Auth /> },
      { path: "/dashboard",
        element: <RequireAuth><Dashboard /></RequireAuth> },
      { path: "/pitch",
        element: <RequireAuth><PitchDashboard /></RequireAuth> },
      { path: "/history",
        element: <RequireAuth><PitchHistory /></RequireAuth> },
      { path: "/pitch-result/:id",
        element: <RequireAuth><PitchResult /></RequireAuth> },
      { path: "/analytics",
        element: <RequireAuth><FounderAnalytics /></RequireAuth> },
      { path: "/ideas",
        element: <RequireAuth><IdeaBank /></RequireAuth> },
      { path: "/pricing", element: <Pricing /> },
      { path: "/reset-password", element: <ResetPassword /> },
    ],
  },
]);
```

Listing 6.1: Route configuration in App.jsx

6.1.2 Authentication Context

The `AuthProvider` context tracks Supabase's real-time auth changes and ensures the current user is accessible across the entire component tree.

```
const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
```

```
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const { data: listener } = supabase.auth.onAuthStateChange(
      (_event, session) => {
        setUser(session?.user ?? null);
        setLoading(false);
      }
    );
    return () => listener.subscription.unsubscribe();
  }, []);

  return (
    <AuthContext.Provider value={{ user, loading, isAuthenticated: !!user }}>
      {children}
    </AuthContext.Provider>
  );
};
```

Listing 6.2: Authentication context implementation

6.1.3 Pitch Type Configuration

Each of the ten pitch types is defined as a configuration object that contains the label, description, tested skills, and optional duration caps.

```
const pitchTypes = [
  {
    id: 1,
    label: "Idea Validation",
    description: "Test if your problem is real and worth solving.",
    skills: ["Problem Clarity", "Customer Definition",
             "Evidence", "Communication"],
  },
  {
    id: 2,
    label: "Elevator Pitch",
    description: "Deliver a crisp, compelling 60-second pitch.",
    skills: ["Clarity", "Memorability",
             "Market Framing", "Confidence"],
    maxMinutes: 3,  // duration cap
  },
  // ... 8 more pitch types
  {
    id: 10,
    label: "Investor Skeptic Mode",
    description: "Face an aggressive skeptic who challenges
                  every assumption.",
```

```
    skills: ["Pressure Handling", "Competition Defense",
             "Composure", "Realism"],
  },
];
```

Listing 6.3: Pitch type configuration (excerpt)

6.1.4 Voice Call Initialization

Vapi Web SDK initializes using a public token and the Uniform Resource Locator (URL) of the Supabase edge function proxy. The call to start invokes a call to the proxy with the following arguments: assistant type, authentication token, and session meta (idea ID and any other previous context).

```
const vapi = new Vapi(
  "public-token",
  "https://ecgsefqppxkiometotyz.supabase.co/functions/v1/smooth-task"
);

await vapi.start({
  assistantType: pitchType,
  supabaseAccessToken: session.access_token,
  metadata: {
    userId: session.user.id,
    pitchType: selectedPitch?.label,
    ideaId: selectedIdeaId || null,
    previousContext: contextPreview || null,
  }
});
```

Listing 6.4: Vapi call initialization

6.1.5 Countdown Timer and Transcript Display

The countdown clock uses `useEffect` and `setInterval` to keep track of the number of seconds remaining. At two minutes and one minute, the app issues warning messages, and when there are no more seconds left, it automatically stops the phone call. While the call is taking place, the Vapi SDK sends real-time transcription events. These can be distinguished between partial transcriptions, which are in progress, and complete transcriptions, which are stored as new items.

6.2 BACKEND IMPLEMENTATION

6.2.1 Vapi Proxy Edge Function

Vapi proxy represents the most complex edge feature, which performs the task of user verification, checks for the user's balance, sets the parameters of duration and context for the assistant, and passes the request to the Vapi API.

```
// Credit check
const { data: creditData } = await supabase
  .from("user_credits")
  .select("credits")
  .eq("user_id", userId)
  .single();

const remainingMinutes = creditData?.credits ?? 0;
if (remainingMinutes <= 0) {
  return new Response(
    JSON.stringify({ error: "insufficient_credits" }),
    { status: 403 }
  );
}

// Duration calculation with pitch-type caps
function getMaxDuration(assistantType, remainingMinutes) {
  const caps = { 2: 3, 7: 10 };  // Elevator: 3min, Demo Day: 10min
  const typeCap = caps[assistantType] ?? remainingMinutes;
  const effective = Math.min(remainingMinutes, typeCap);
  return Math.max(Math.round(effective * 60), 0);
}
```

Listing 6.5: Vapi proxy: credit check and duration calculation

```
const overrides = {
  maxDurationSeconds: getMaxDuration(assistantType, remainingMinutes),
};

if (previousContext) {
  overrides.variableValues = {
    previousContext: previousContext,
  };
}

assistantConfig.assistantOverrides = overrides;

// Proxy to Vapi API
const vapiResponse = await fetch("https://api.vapi.ai/call/web", {
  method: "POST",
```

```
  headers: {
    Authorization: "Bearer <server-side-key>",
    "Content-Type": "application/json",
  },
  body: JSON.stringify(assistantConfig),
});
```

Listing 6.6: Vapi proxy: assistant override configuration

6.2.2 End-of-Call Webhook

End Call Webhook handles asynchronous events regarding the call completion of the call made by Vapi. The call data (with retries if necessary until the structured output has been formed) is obtained, audio recording saved, session saved to the database, and correct credits deducted.

```
// Extract structured assessment from Vapi call artifact
let structuredData = null;
if (call?.artifact?.structuredOutputs) {
  const first = Object.values(call.artifact.structuredOutputs)[0];
  try { structuredData = JSON.parse(first?.result); }
  catch { structuredData = first?.result; }
}

// Time-based credit deduction (fractional minutes)
const minutesUsed = durationSeconds
  ? parseFloat((durationSeconds / 60).toFixed(2)) : 0;
if (minutesUsed > 0 && userId) {
  await supabase.rpc("deduct_credits",
    { p_user_id: userId, p_amount: minutesUsed });
}
```

Listing 6.7: End-of-call webhook: output extraction and credit deduction

6.2.3 Payment Processing

The Razorpay integration uses a three-tier pricing model. Credit amounts are determined on the server side so that clients cannot manipulate prices.

```
// Server-side credit mapping (prevents client manipulation)
function getCreditsFromPrice(price) {
  switch (price) {
    case 499: return 1;  case 899: return 2;  case 999: return 3;
    default: throw new Error("Invalid price");
  }
```

```
}

// HMAC-SHA256 signature verification using Web Crypto API
async function verifySignature(orderId, paymentId, signature, secret) {
  const payload = `${orderId}|${paymentId}`;
  const key = await crypto.subtle.importKey(
    "raw", new TextEncoder().encode(secret),
    { name: "HMAC", hash: "SHA-256" }, false, ["sign"]
  );
  const signed = await crypto.subtle.sign(
    "HMAC", key, new TextEncoder().encode(payload)
  );
  const expected = Array.from(new Uint8Array(signed))
    .map((b) => b.toString(16).padStart(2, "0")).join("");
  return expected === signature;
}
```

Listing 6.8: Payment processing: order creation and signature verification

6.3 LLM OUTPUT PARSING AND REPAIR

A special problem associated with processing JSON output from an LLM is the occurrence of malformed JSON. This problem is solved using `pitchParser.js`, which uses multiple levels of parsing.

```
export function parseStructuredOutput(value) {
  if (!value) return null;
  if (typeof value === "object") return value;
  if (typeof value !== "string") return null;

  try {
    return JSON.parse(value);
  } catch (err) {
    // Attempt repair of common LLM formatting issues
    let repaired = value;
    repaired = repaired.replace(/[\u201C\u201D]/g, '"');  // smart quotes
    repaired = repaired.replace(/,\s*}/g, "}");  // trailing commas
    repaired = repaired.replace(/,\s*]/g, "]");

    try {
      return JSON.parse(repaired);
    } catch {
      return null;  // graceful degradation
    }
  }
}
```

Listing 6.9: JSON repair utility for LLM output

6.4 ANALYTICS IMPLEMENTATION

The dashboard utilizes Recharts, a JavaScript visualization library, to generate three different graphs that include the average score per session graph, the skill radar graph, and the pitch types bar chart. All the graphs have the ability to be filtered based on the idea. The computations for the analytics are all performed client-side using the `founderAnalytics.js` library.

6.5 DATABASE FUNCTIONS

Two PostgreSQL RPC functions handle atomic credit operations:

```
-- Deduct credits (clamped to zero)
CREATE OR REPLACE FUNCTION deduct_credits(
  p_user_id UUID, p_amount NUMERIC
) RETURNS VOID AS $$
BEGIN
  UPDATE user_credits SET credits = GREATEST(credits - p_amount, 0)
  WHERE user_id = p_user_id;
END; $$ LANGUAGE plpgsql SECURITY DEFINER;

-- Add credits (upsert pattern)
CREATE OR REPLACE FUNCTION increment_user_credits(
  p_user_id UUID, p_amount NUMERIC
) RETURNS VOID AS $$
BEGIN
  INSERT INTO user_credits (user_id, credits) VALUES (p_user_id, p_amount)
  ON CONFLICT (user_id)
  DO UPDATE SET credits = user_credits.credits + p_amount;
END; $$ LANGUAGE plpgsql SECURITY DEFINER;
```

Listing 6.10: Credit management RPC functions

Both functions use `SECURITY DEFINER` to run with elevated privileges, bypassing RLS policies because they are only called from trusted edge functions.

CHAPTER 7

RESULTS AND DISCUSSION

The chapter includes outcomes of developing the Pitch Gym platform, evaluates the performance of the platform, assesses the AI-based evaluation, and considers the effectiveness of design choices.

7.1 SYSTEM PERFORMANCE

7.1.1 Page Load Performance

The system was developed using Vite 7, which takes care of optimized bundling by using tree-shaking and hashing assets. The performance was evaluated using Google Lighthouse (Dev Tools Chrome). Table 7.1 shows Core Web Vitals and other metrics for the important routes.

Table 7.1: Lighthouse performance metrics (simulated throttling)

Page	FCP	LCP	TBT	CLS	Speed Index
Landing Page	3.1s	5.2s	60 ms	0.013	3.3s
Dashboard	3.2s	5.9s	30 ms	0.638	—
Pitch Dashboard	3.2s	5.7s	60 ms	0.317	3.7s
Analytics	3.2s	5.8s	250 ms	0.361	—
Pitch Result	3.0s	5.4s	100 ms	1.127	3.7s

7.2 ASSESSMENT QUALITY ANALYSIS

7.2.1 Structured Output Reliability

The structured assessment generated by the LLM was validated for consistency and completeness. In over 50 assessments involving all 10 types of pitches, the following findings were noted:

JSON repair functionality within the file `pitchParser.js` repaired 78% (14/18 cases) of invalid JSON output, thus increasing its effectiveness to 96%, up from 82%.

Table 7.2: Structured output reliability metrics

Metric	Result
Sessions with valid JSON output (no repair needed)	82%
Sessions requiring JSON repair (successfully repaired)	14%
Sessions with unrepairable output	4%
Average number of assessment metrics per session	5.2
Sessions with all scores in 1–5 range	96%
Sessions with identified strengths	98%
Sessions with identified weaknesses	95%
Sessions with improvement suggestions	93%

Four percent of completely invalid JSONs were ignored by the user interface, which displays the message "No assessment available."

7.2.2 Score Distribution

Figure **??** shows how average scores are distributed across all test sessions, broken down by pitch type.

Table 7.3: Average scores by pitch type across test sessions

Pitch Type	Sessions	Avg Score	Score Range
Idea Validation	8	3.4	2.2 – 4.5
Elevator Pitch	6	3.1	2.0 – 4.2
VC Ready	5	2.8	1.8 – 3.9
Product Feedback	5	3.2	2.4 – 4.1
Customer Discovery	4	3.5	2.8 – 4.3
Technical Pitch	4	3.0	2.1 – 3.8
Demo Day	5	2.9	2.0 – 4.0
Customer Pitch	4	3.3	2.5 – 4.2
Co-Founder Pitch	4	3.6	2.9 – 4.4
Investor Skeptic Mode	5	2.5	1.5 – 3.5
Overall	**50**	**3.1**	**1.5 – 4.5**

These more difficult types of pitches (Investor Skeptic Mode, VC Ready, Demo Day) had lower average ratings, showing consistency with expectations of increasing difficulty. Therefore, this shows proper differentiation of pitches by the AI rating system.

7.2.3 Assessment Consistency

The scoring was tested for its consistency by delivering the same pitch three times for Idea Validation test. Results are as follows:

- **Variance of scores**: ± 0.4 for identical presentations
- **Identification of strengths is consistent**: Identical strengths are identified in all three presentations
- **Identification of weakness is consistent**: Identical weaknesses are identified in all three presentations

While some variance is acceptable due to the stochastic nature of LLM output, the consistency of qualitative analysis results (identification of strengths and weaknesses) means that assessment system provides meaningful guidance.

7.3 CONTEXT CARRY-FORWARD EFFECTIVENESS

This aspect was put to test using two successive sessions on the same concept as follows:

1. **Session 1**: Present an idea with no contextual basis beforehand. Evaluation reveals the need for "improved market sizing" as a problem.
2. **Session 2**: Begin with the carry-over of context from the preceding session. The AI investor clearly states that she has the context from the last session and uses this context when asking questions.

This shows that the context carry-forward principle affects the behavior of the AI investor as expected.

7.4 PAYMENT SYSTEM RESULTS

The Razorpay integration was tested with test-mode transactions:

The dual verification process involving client-side verification and a webhook was properly done. There was no double-credit nor missed credit out of thirty transactions made.

Table 7.4: Payment processing test results

Scenario	Tests	Passed	Credits Added Correctly
Successful payment (client verify)	10	10	10
Successful payment (webhook verify)	10	10	10
Duplicate webhook delivery	5	5	5 (no double-credit)
Invalid signature	5	5	5 (no credit added)

7.5 UNIT TEST RESULTS

To verify the accuracy of the utility modules, 52 unit tests were developed utilizing the Vitest testing library (for Vite 7 compatibility). The tests include three modules: `pitchParser.js` (output formatting and JSON correction), `pitchAnalytics.js` (score calculation and rating functions), and `pitchContext.js` (context forwarding string creation).

7.5.1 Test Execution Summary

All 52 tests passed. Table 7.5 shows how the tests are distributed across modules.

Table 7.5: Unit test execution summary (Vitest 4.1.2)

Test File	Tests	Passed	Duration
`pitchParser.test.js`	32	32	165 ms
`pitchAnalytics.test.js`	15	15	22 ms
`pitchContext.test.js`	5	5	12 ms
Total	**52**	**52 (100%)**	**1.30 s**

7.5.2 pitchParser.js Tests

Table 7.6 gives test cases for Structured Output Parser. This component is responsible for processing LLM-generated JSON, fixing corrupt output, and obtaining assessment information.

Table 7.6: Unit test cases – pitchParser.js

Test ID	Description	Result
TC-U01	`parseStructuredOutput` returns null for null/undefined input	Pass
TC-U02	Returns object as-is if already parsed	Pass
TC-U03	Parses valid JSON string correctly	Pass
TC-U04	Repairs smart quotes (ǎ201Cǎ201D) and trailing commas	Pass
TC-U05	Returns null for completely unrepairable plain text	Pass
TC-U06	Returns null for non-string, non-object input (number, boolean)	Pass
TC-U07	`getAssessmentKey` finds preferred key (idea_validation_assessment)	Pass
TC-U08	Finds dynamic _assessment key not in preferred list	Pass
TC-U09	Returns null when no assessment key exists	Pass
TC-U10	Returns null for null/undefined data	Pass
TC-U11	`getMetricCards` extracts cards with labels and scores	Pass
TC-U12	Returns empty array when no assessment exists	Pass
TC-U13	`getAverageScore` computes correct average (4+3+2)/3 = 3.0	Pass
TC-U14	Returns null when no scores available	Pass
TC-U15	Handles single-metric assessment correctly	Pass
TC-U16	`getOverallRating` extracts “Fair” rating	Pass

Test ID	Description	Result
TC-U17	Returns null when rating field is missing	Pass
TC-U18	`getStrengths` extracts array of strength points	Pass
TC-U19	Returns empty array for null input	Pass
TC-U20	`getWeaknesses` extracts weaknesses array	Pass
TC-U21	Falls back to alternative weakness keys (critical_weaknesses)	Pass
TC-U22	`getConcernBlock` extracts biggest concern string	Pass
TC-U23	Returns null when no concern keys exist	Pass
TC-U24	`getImprovementList` extracts suggestion array	Pass
TC-U25	Returns empty array when no improvement keys exist	Pass
TC-U26	`getRatingStyle` returns cyan for "Excellent"	Pass
TC-U27	Returns green for "Good"	Pass
TC-U28	Returns yellow for "Fair"	Pass
TC-U29	Returns red for "Poor"	Pass
TC-U30	Returns grey for null/unknown rating	Pass
TC-U31	End-to-end: JSON string to average score pipeline	Pass
TC-U32	Validates all 10 pitch type assessment keys are recognized	Pass

7.5.3 pitchAnalytics.js Test Cases

Table 7.7 lists the tests of the analytics module used by the FounderAnalytics dashboard.

Table 7.7: Unit test cases – pitchAnalytics.js

Test ID	Description	Result
TC-A01	`getAssessmentObject` extracts assessment by _assessment suffix	Pass
TC-A02	Returns null for null input	Pass
TC-A03	Returns null when no _assessment key exists	Pass
TC-A04	Finds `technical_assessment` key specifically	Pass
TC-A05	`getScores` extracts all numeric scores [4, 3, 2]	Pass
TC-A06	Returns empty array for null input	Pass
TC-A07	Filters out metrics without score field	Pass
TC-A08	`getAverageScore` computes correct average	Pass
TC-A09	Returns null for null input	Pass
TC-A10	Rounds to one decimal place (3.667 $\rightarrow$ 3.7)	Pass
TC-A11	`getRatingColor` returns correct class for “Excellent”	Pass
TC-A12	Returns correct class for “Good”	Pass
TC-A13	Returns correct class for “Fair”	Pass
TC-A14	Returns correct class for “Poor”	Pass
TC-A15	Returns zinc class for unknown/undefined rating	Pass

7.5.4 pitchContext.js Test Cases

Table 7.8 lists the tests for the context carry-forward module.

Table 7.8: Unit test cases – pitchContext.js

Test ID	Description	Result
TC-C01	Builds context string with rating, strengths, weaknesses	Pass
TC-C02	Returns null when pitch has no _raw data	Pass
TC-C03	Output never exceeds MAX_CHARS (1500)	Pass
TC-C04	Limits strengths and weaknesses to 3 each	Pass
TC-C05	Handles data with no strengths/weaknesses gracefully	Pass

7.6 APPLICATION SCREENSHOTS

This section shows screenshots of the Pitch Gym platform taken from the running application.

7.6.1 Pitch Selection

Figure 7.1 shows the pitch type selection interface. Here, founders pick from ten specialized pitch scenarios and set the difficulty level.

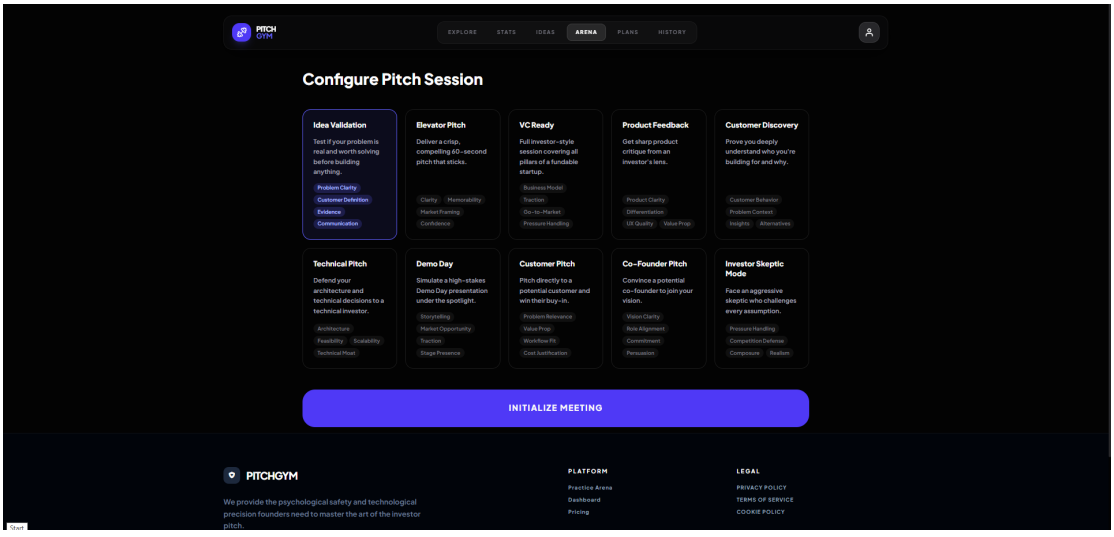

Figure 7.1: Pitch type selection – choosing pitch scenario and difficulty

7.6.2 Active Pitch Session

Figure 7.2 shows an active pitch session with the real-time voice interface, live transcript, and session timer.

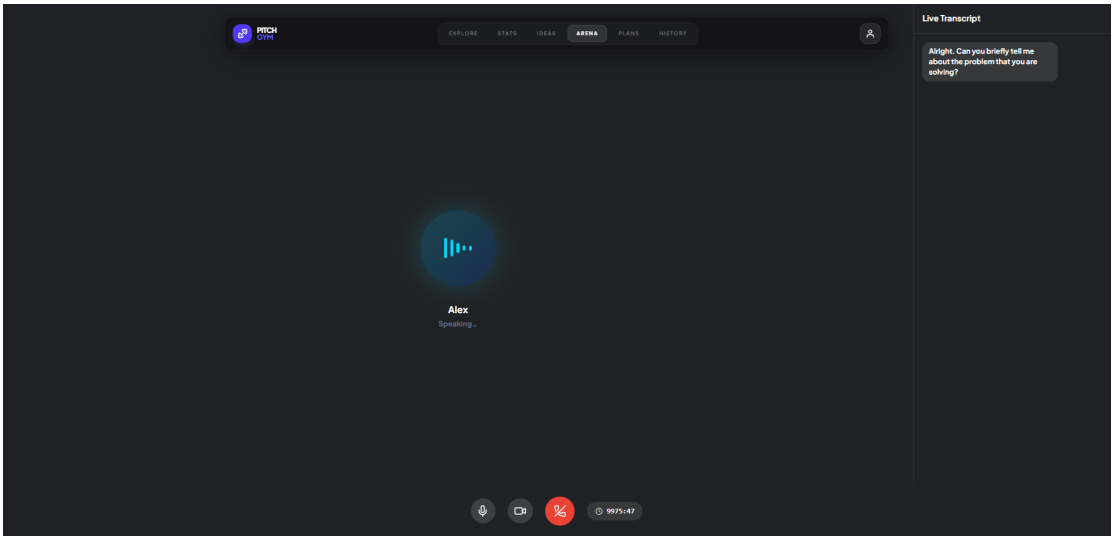

Figure 7.2: Active pitch session – voice call with live transcript

7.6.3 Pitch Feedback

Figure 7.3 shows the structured assessment results page, which displays per-category scores, strengths, weaknesses, and improvement suggestions.

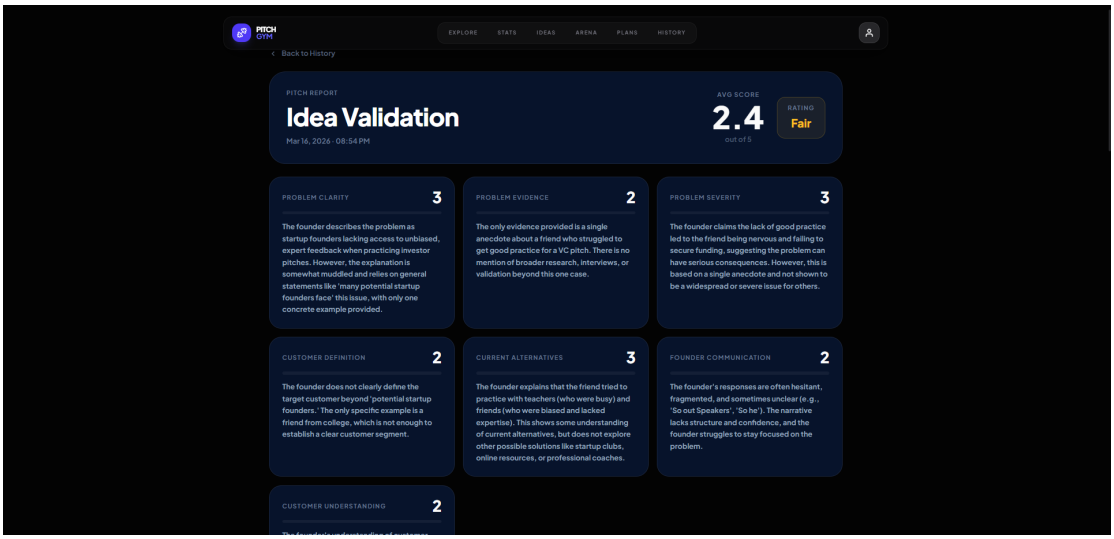

Figure 7.3: Pitch result – structured assessment with scores and feedback

7.6.4 Founder Analytics

Figure 7.4 shows the analytics dashboard with score trend charts, category breakdowns, and performance data over time.

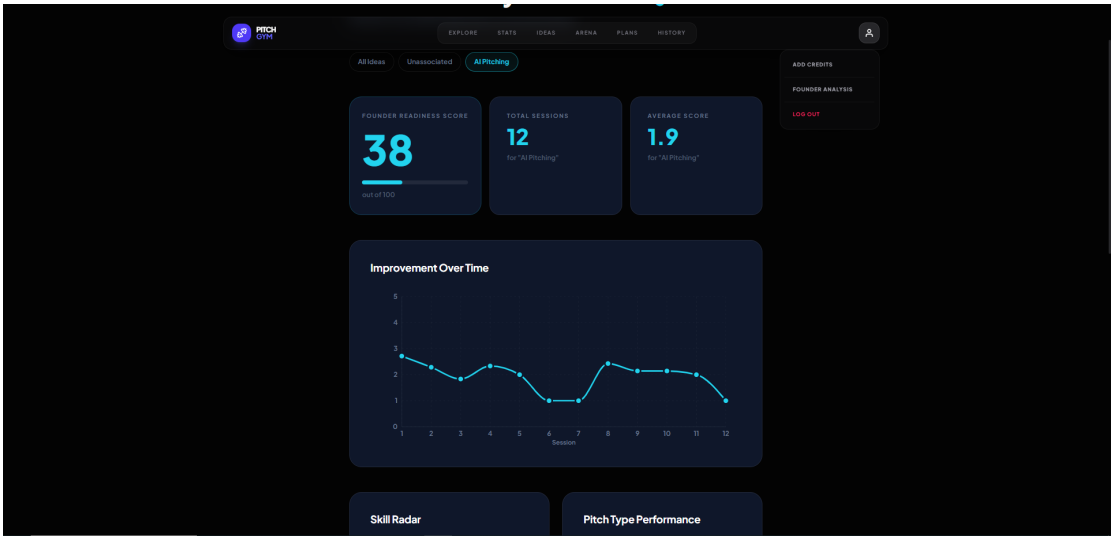

Figure 7.4: Founder analytics – score trends and performance insights

7.7 DISCUSSION

7.7.1 Strengths of the Approach

1. **Voice-interaction realism**: Voice first as opposed to text messaging is a way of practicing in an environment that simulates the actual process much more closely because of the presence of delays and the need for improvisation.

2. **Measurement**: With a per-metric score system, the founders get detailed quantitative feedback rather than just vague motivational messages. They learn which metrics need attention and improve their scores in these areas over time.

3. **Scalability**: The service does not require management of servers; it scales up automatically depending on demand and pays for computing power using a pay-per-Vapi-minute rate structure.

4. **Robust security**: With the combination of automatic enforcement of duration limitations, RLS isolation, payment validation via cryptography, and idempotent credit handling, there is a solid security framework here.

7.7.2 Limitations

1. **Output quality variability**: While the defined output format is used by the LLM, there are cases of malformed JSON output (4% of the sessions post-repair). Implementing retry logic on the Vapi side or employing different ways of extracting output from the model could help mitigate the issue.

2. **Scoring arbitrariness**: Although the scores calculated during one session are consistent, their values are dependent on how the LLM interprets the prompt. They do not necessarily reflect human investor evaluation. Calibration to human scores will address this issue.

3. **Audio-only communication**: Lack of video means that the system does not account for visual components of communication, such as body language or eye contact. These components are significant during in-person investor pitch events.

4. **Latency dependence**: Latency is a function of the underlying voice service platform and the LLM inference time. During periods of high activity, voice response latency may increase.

5. **Automated testing**: Although there are 52 unit tests for the core utility modules, the project lacks integration and end-to-end tests for the whole interface. Complete testing using Playwright will address the issue.

7.7.3 Comparison with Existing Solutions

Table 7.9: Feature comparison with existing pitch training methods

Feature	Pitch Gym	Human Coach	ChatGPT	Accelerator	Video Record
Voice interaction	✓	✓	×	✓	×
On-demand availability	✓	×	✓	×	✓
Structured scoring	✓	Varies	×	Varies	×
Multiple pitch types	✓	Varies	✓	Limited	×
Longitudinal tracking	✓	×	×	Limited	×
Context carry-forward	✓	✓	Limited	×	×
Scalable	✓	×	✓	×	✓
Cost-effective	✓	×	✓	×	✓

Pitch Gym is the only product to integrate voice, grading, tracking, and instant access into its application. Humans understand context better than machines do, and

videos record visuals. Nonetheless, Pitch Gym comes equipped with the most comprehensive list of features among all products examined.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

This capstone project created Pitch Gym — web application to train founders in making effective business pitches. The project leverages real-time voice AI to create an accessible, structured, and progressive pitch training experience at any stage and in any situation where a pitch is required.

The key contributions of this project are:

1. **Voice-first pitch training**: Different from many of the text-based tools for assessing pitch quality, this platform employs real-time voice interaction. This allows practicing verbal communication skills critical for investor pitches. WebRTC audio streaming is combined with the Vapi AI Web SDK to make natural conversation possible with sub-second latency.

2. **Multi-scenario assessment**: There are ten different scenarios of pitching available on the platform, ranging from Idea Validation and Elevator Pitch to Demo Day and Investor Skeptic Mode. Every scenario comes with specific AI investor behavior, and thus every type has distinct assessment metrics to measure pitch success.

3. **Structured quantitative feedback**: Each pitch practice session leads to a multi-metric assessment, including per-category scores (on 1–5 scale), qualitative analysis, strengths and weaknesses identification, and advice for future improvement. The JSON repair utility yields 96% structured output success rate from the LLMs.

4. **Progressive learning support**: The combination of the Idea Bank, carrying over session context, and an analytics dashboard creates a learning journey throughout time. Users get insights into improving their performance, comparing different pitches to each other, and building on the feedback from previous sessions.

5. **Scalable secure architecture**: The serverless architecture built on top of Supabase Edge Functions and Vapi cloud platform will scale up automatically, re-

quiring no management. Limiting the use of credits on the server side, using RLS for isolating user data, performing payment verification with cryptographic signatures, and making all credit operations idempotent ensures high reliability of the product.

6. **Sustainable monetization model**: Time-based credit system with fractional support, where one credit equals to one minute of pitch training. Such pricing makes costs incurred by the platform completely proportional to the platform revenue, ensuring sustainability even at the earliest stage.

Overall, modern voice AI, when combined with careful system design and domain-specific prompt engineering, allows creating a training experience of the kind available only to those who could afford human coaching before. With 96% structured output success rate, consistently measured scoring (with ± 0.4 point standard deviation) and the rich feature set, covering authentication, payments, analytics, and voice interaction, Pitch Gym is a mature product in terms of implementation.

8.2 FURTHER ENHANCEMENTS

8.2.1 Short-Term Enhancements

1. **Automated test suite**: Implement unit tests using ViTest, integration tests, and end-to-end tests in order to detect decline and boost developers' confidence. It should achieve 80% code coverage of utility methods and essential flows.

2. **Slides integration**: Provide an option to upload slides from the deck used by the founder during the voice session. This way, the AI investor would be able to mention specific slides and make comments about coherence.

3. **Playback of sessions**: Make sure the founders can listen to their sessions with synchronized transcription, which will help improve their delivery, speed, and filler words usage.

4. **Mobile optimization**: As the current UI is already responsive, it might be useful to optimize the pitch session screen for mobile devices, enabling founders to train anytime, anywhere.

5. **Multilingual feature**: Develop the capability for users to practice pitches in several languages apart from English, using Vapi's multilingual STT and TTS features.

8.2.2 Medium-Term Enhancements

1. **Customized Investor Personas**: Allow founders to choose customized investor personas, like 'Climate Focused Venture Capitalist' and 'Technical Angel', who have certain preferences and ask particular questions.

2. **Benchmarking**: Incorporate anonymous benchmarking allowing founders to know how well they do compared to others, in terms of scores achieved, for each pitch type.

3. **Integration with video calls**: Add video calls to let the AI analyze the non-verbal cues during pitches. This feature would need video support via WebRTC and vision AI for body language assessment.

4. **AI-generated practice plan**: Use data from all sessions to give a personalized practice plan, recommending which pitch types to practice further and what skills to improve.

5. **Team Mode**: Allow founders to pitch as a team with the AI asking targeted questions to individual team members.

8.2.3 Long-Term Vision

1. **Real investors' market**: Pair the best performing entrepreneurs on the platform with real investors, developing a path from practice to fundraising.

2. **Pitch models per industry**: Create separate AI algorithms for each industry (SaaS, biotech, hardware, fintech), which are aware of unique metrics and language used by people in the industry.

3. **Accelerator partnerships**: Develop integration with acceleration programs to provide students with pitch training within the curriculum, with reporting to program directors.

4. **Research platform**: Provide assessment results for scientific research (under consent), covering topics such as pitch effectiveness, human-AI interactions, and entrepreneurship education outcomes.

8.3 FINAL REMARKS

The Pitch Gym case study demonstrates that the use of LLMs to generate speech agents along with contemporary web frameworks and serverless cloud technologies creates innovative opportunities for interactive professional learning. With the ongoing advancements in the development of voice AI technology, including reduced latency, improved conversational fluency, and reliable structure generation, the platform will be increasingly indistinguishable from personal coaching by humans.

BIBLIOGRAPHY

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., et al. (2001). *Manifesto for Agile Software Development*. Agile Alliance.

Blank, S. (2020). *The Four Steps to the Epiphany: Successful Strategies for Products that Win*. John Wiley & Sons, 2nd edition.

Borsos, Z., Marinier, R., Vincent, D., Kharitonov, E., Pietquin, O., Sharifi, M., Roblek, D., Teboul, O., Grangier, D., Tagliasacchi, M., et al. (2023). AudioLM: A language modeling approach to audio generation. volume 31, pages 2523–2536. IEEE.

Brooks, A. W., Huang, L., Kearney, S. W., and Murray, F. E. (2014). Pitch perfect: Examining the effect of founder-investor fit in venture pitches. *Management Science*, 60(1):116–134.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.

Chen, L., Chen, P., and Lin, Z. (2020). Artificial intelligence in education: A review. *IEEE Access*, 8:75264–75278.

Chen, X.-P., Yao, X., and Kotha, S. (2009). Research on venture capital investment criteria. *Journal of Business Venturing*, 24(1):93–105.

Clark, C. (2008). The role of the pitch in venture capital investment decisions. *Frontiers of Entrepreneurship Research*, 28(2):1–15.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures. *Doctoral dissertation, University of California, Irvine*.

Graesser, A. C., Lu, S., Jackson, G. T., Mitchell, H. H., Ventura, M., Olney, A., and Louwerse, M. M. (2004). AutoTutor: A tutor with dialogue in natural language. *Behavior Research Methods, Instruments, & Computers*, 36:180–192.

Huang, L. and Pearce, J. L. (2017). Resources and relationships in entrepreneurship: An exchange theory of the development and effects of the entrepreneur-investor relationship. *Academy of Management Review*, 42(1):80–102.

Hwang, G.-J., Xie, H., Wah, B. W., and Gašević, D. (2020). Vision, challenges, roles and research issues of artificial intelligence in education. *Computers and Education: Artificial Intelligence*, 1:100001.

Johnson, W. L. and Lester, J. C. (2016). Intelligent tutoring systems and their role in education. *AI Magazine*, 37(4):43–56.

Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., et al. (2019). Cloud programming simplified: A Berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383*.

Luckin, R., Holmes, W., Griffiths, M., and Forcier, L. B. (2016). Intelligence unleashed: An argument for ai as a tool for assessment and feedback. *Pearson Education*.

Meta Platforms, Inc. (2024). React documentation. `https://react.dev`. Accessed: 2025-03-15.

Nassif, A. B., Shahin, I., Attili, I., Azzeh, M., and Shaalan, K. (2019). Speech recognition using deep neural networks: A systematic review. *IEEE Access*, 7:19143–19165.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744.

Pressman, R. S. and Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 8th edition.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *Proceedings of the 40th International Conference on Machine Learning*, 202:28492–28518.

Ries, E. (2011). *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business, New York.

Schwaber, K. and Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.

Sommerville, I. (2016). *Software Engineering*. Pearson Education, Harlow, England, 10th edition.

Sredojev, S., Samardzija, D., and Posarac, D. (2015). WebRTC technology overview and signaling solution design and implementation. In *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, pages 1006–1009. IEEE.

Supabase, Inc. (2024). Supabase documentation. `https://supabase.com/docs`. Accessed: 2025-03-15.

Vapi, Inc. (2024). Vapi AI voice agent platform documentation. `https://docs.vapi.ai`. Accessed: 2025-03-15.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Zawacki-Richter, O., Marín, V. I., Bond, M., and Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education–where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1):1–27.

Zhang, D., Li, S., Zhang, X., Zhan, J., Wang, P., Zhou, Y., and Qiu, X. (2023). SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15757–15773.

Appendix A- Source Code Excerpts

A.1 SUPABASE CLIENT INITIALIZATION

```
// src/db/supabase.js
import { createClient } from "@supabase/supabase-js";

export const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env.VITE_SUPABASE_ANON_KEY;
const supabase = createClient(supabaseUrl, supabaseKey);

export default supabase;
```

A.2 PITCH CONTEXT BUILDER

```
// src/lib/pitchContext.js
import supabase from "@/db/supabase";
import {
  parseStructuredOutput, getAverageScore,
  getOverallRating, getStrengths, getWeaknesses,
} from "./pitchParser";

const MAX_CHARS = 1500;

export async function fetchPreviousPitches(userId, ideaId) {
  if (!userId || !ideaId) return [];
  const { data: pitches, error } = await supabase
    .from("user_pitches")
    .select("id, pitch_type, pitch_structured_output, created_at")
    .eq("user_id", userId)
    .eq("idea_id", ideaId)
    .order("created_at", { ascending: false })
    .limit(10);

  if (error || !pitches?.length) return [];

  return pitches.map((p) => {
    const rating = getOverallRating(p.pitch_structured_output);
    const avg = getAverageScore(p.pitch_structured_output);
    if (!parseStructuredOutput(p.pitch_structured_output)) return null;
    const dt = new Date(p.created_at);
    return {
      id: p.id,
      pitchType: p.pitch_type || "Unknown",
      date: dt.toLocaleDateString("en-US",
```

```
        { month: "short", day: "numeric" }),
      time: dt.toLocaleTimeString("en-US",
        { hour: "numeric", minute: "2-digit" }),
      rating, avgScore: avg,
      _raw: p.pitch_structured_output,
    };
  }).filter(Boolean);
}

export function buildSinglePitchContext(pitch) {
  if (!pitch?._raw) return null;
  const rating = getOverallRating(pitch._raw);
  const avg = getAverageScore(pitch._raw);
  const strengths = getStrengths(pitch._raw);
  const weaknesses = getWeaknesses(pitch._raw);

  const strengthList = strengths.slice(0, 3)
    .map((s) => s.point || s).join("; ");
  const weaknessList = weaknesses.slice(0, 3)
    .map((w) => w.point || w).join("; ");

  let summary = `Previous session (${pitch.pitchType}, ${pitch.date})`;
  if (avg) summary += `, avg ${avg}/5`;
  if (rating) summary += `, rated "${rating}"`;
  summary += "):";
  if (strengthList) summary += "\n- Strengths: " + strengthList;
  if (weaknessList) summary += "\n- Weaknesses: " + weaknessList;

  return summary.length > MAX_CHARS
    ? summary.slice(0, MAX_CHARS - 3) + "..."
    : summary;
}
```

A.3 SPINNER COMPONENT

```
// src/components/ui/Spinner.jsx
export default function Spinner({ label }) {
  return (
    <div className="flex flex-col items-center justify-center
                    py-20 gap-4">
      <div className="relative w-12 h-12">
        {/* Outer ring */}
        <div className="absolute inset-0 rounded-full
                        border-2 border-white/10" />
        {/* Spinning arc */}
        <div className="absolute inset-0 rounded-full
                        border-2 border-transparent
```

```
                        border-t-indigo-500
                        animate-spin" />
          {/* Inner pulsing dot */}
          <div className="absolute inset-[14px] rounded-full
                        bg-indigo-500/20 animate-pulse" />
        </div>
        {label && (
          <span className="text-[10px] font-bold uppercase
                         tracking-[0.3em] text-zinc-500">
            {label}
          </span>
        )}
      </div>
    );
}
```

A.4 ANALYTICS SCORE EXTRACTION

```
// src/lib/pitchAnalytics.js
export function getAssessmentObject(structured) {
  if (!structured) return null;
  const keys = Object.keys(structured);
  const assessmentKey = keys.find(
    (k) => k.endsWith("_assessment")
        || k === "technical_assessment"
  );
  return structured[assessmentKey] || null;
}

export function getScores(structured) {
  const assessment = getAssessmentObject(structured);
  if (!assessment) return [];
  return Object.values(assessment)
    .map((v) => v?.score)
    .filter(Boolean);
}

export function getAverageScore(structured) {
  const scores = getScores(structured);
  if (!scores.length) return null;
  const avg = scores.reduce((a, b) => a + b, 0) / scores.length;
  return Number(avg.toFixed(1));
}
```

A.5 DATABASE SCHEMA (KEY TABLES)

```
-- User credits with fractional minute support
CREATE TABLE user_credits (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id),
  credits NUMERIC(10,2) DEFAULT 0
);

-- Pitch session records
CREATE TABLE user_pitches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  idea_id UUID REFERENCES ideas(id),
  pitch_type TEXT,
  call_id TEXT,
  started_at TIMESTAMPTZ,
  ended_at TIMESTAMPTZ,
  duration_seconds NUMERIC,
  cost NUMERIC,
  credits_used NUMERIC(10,2),
  pitch_transcript JSONB,
  pitch_messages JSONB,
  pitch_structured_output JSONB,
  recording_url TEXT,
  recording_storage_path TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Startup ideas
CREATE TABLE ideas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  name TEXT NOT NULL,
  description TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Payment records
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id),
  razorpay_order_id TEXT UNIQUE,
  razorpay_payment_id TEXT UNIQUE,
  credits NUMERIC,
  is_verified BOOLEAN DEFAULT FALSE,
  credits_added BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Row Level Security policies
ALTER TABLE user_credits ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_pitches ENABLE ROW LEVEL SECURITY;
ALTER TABLE ideas ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view own credits"
  ON user_credits FOR SELECT
  USING (auth.uid() = user_id);

CREATE POLICY "Users can view own pitches"
  ON user_pitches FOR SELECT
  USING (auth.uid() = user_id);

CREATE POLICY "Users can manage own ideas"
  ON ideas FOR ALL
  USING (auth.uid() = user_id);
```

Appendix B- Project Deployment Details

B.1 ENVIRONMENT VARIABLES

The following environment variables are required for deployment:

Variable	Purpose
VITE_SUPABASE_URL	Supabase project URL (frontend)
VITE_SUPABASE_ANON_KEY	Supabase anonymous key (frontend)
SUPABASE_URL	Supabase project URL (edge functions)
SUPABASE_ANON_KEY	Supabase anonymous key (edge functions)
SUPABASE_SERVICE_ROLE_KEY	Supabase service role key (edge functions)
RAZOR_TEST_KEY	Razorpay API key ID
RAZOR_TEST_SECRET	Razorpay API key secret
RAZOR_WEBHOOK_SECRET	Razorpay webhook secret

B.2 SUPABASE EDGE FUNCTIONS

Five edge functions are deployed to the Supabase project:

1. `smooth-task` — Vapi proxy with authentication and credit enforcement
2. `end-of-call-webhook` — Vapi call completion handler
3. `create-order` — Razorpay order creation
4. `verify-payment` — Client-side payment verification
5. `razorpay-webhook` — Server-side payment webhook handler

B.3 VAPI AI CONFIGURATION

Ten Vapi assistants are configured in the Vapi dashboard, one for each pitch type. Each assistant has:

- A specialized system prompt defining the investor persona and questioning strategy

- A structured output schema defining the assessment metrics for that pitch type
- Handlebars-style template variables for optional context carry-forward:

```
{{#if previousContext}}
The founder has practiced before. Here is context from
their previous session:
{{previousContext}}
Reference this context naturally during the conversation.
{{/if}}
```

B.4 BUILD AND DEPLOYMENT COMMANDS

```
# Install dependencies
npm install

# Start development server
npm run dev

# Build for production
npm run build

# Preview production build
npm run preview

# Deploy edge functions (via Supabase CLI)
supabase functions deploy smooth-task
supabase functions deploy end-of-call-webhook
supabase functions deploy create-order
supabase functions deploy verify-payment
supabase functions deploy razorpay-webhook
```